

ПАРАЛЛЕЛИЗМ НА УРОВНЕ ИНСТРУКЦИЙ

Архитектура процессоров и технология CUDA

Перцев Дмитрий – 04.10.2026

Содержание лекции



Часть I. Конвейерная обработка

1. Конвейер и его характеристики
2. Конфликты (hazards) и их разрешение

Часть II. Динамическое планирование

1. Алгоритм Томасуло
2. Пример работы и таблица состояний

Часть III. Суперскалярные процессоры

1. Статические и динамические суперскалярные процессоры
2. Sun UltraSPARC III и IBM Power 6

Часть IV. Спекулятивное исполнение



1. Спекулятивный суперскалярный процессор (ССП)
2. Буфер упорядочивания (ROB)

Часть V. VLIW и EPIC

1. Архитектура VLIW
2. EPIC и архитектура IA-64

КОНВЕЙЕР И ЕГО ХАРАКТЕРИСТИКИ

Конвейер



- Характеристики:
  - латентность
  - пропускная способность
- Не конвейер
  - для быстрого выполнения малого количества операций
- Конвейер
  - для быстрого выполнения большого количества повторяющихся операций
  - время выполнения одной операции больше чем у не конвейерной реализации

Исторические примеры



Рис. 1. **UNIVAC LARC (1960 год, США)**



Рис. 2. **IBM 7030 Stretch (1961 г.)**

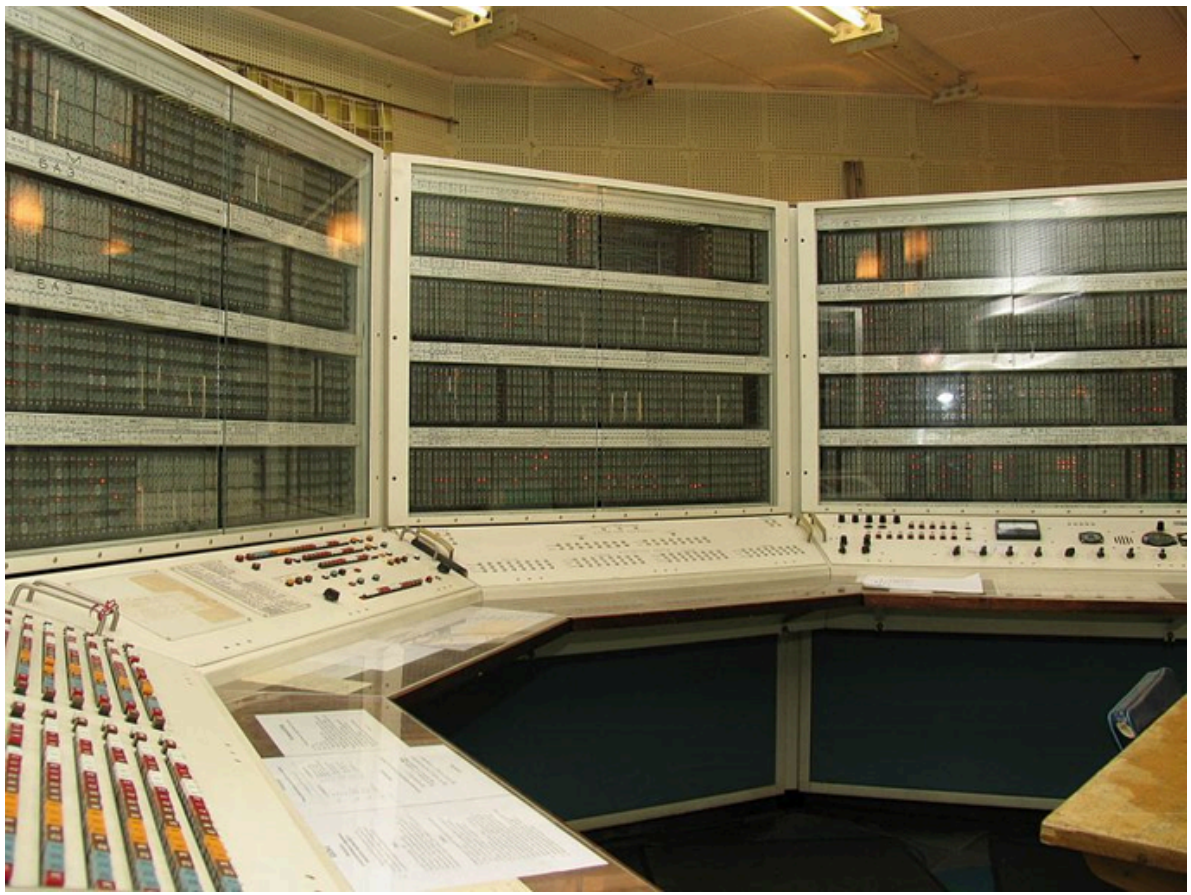
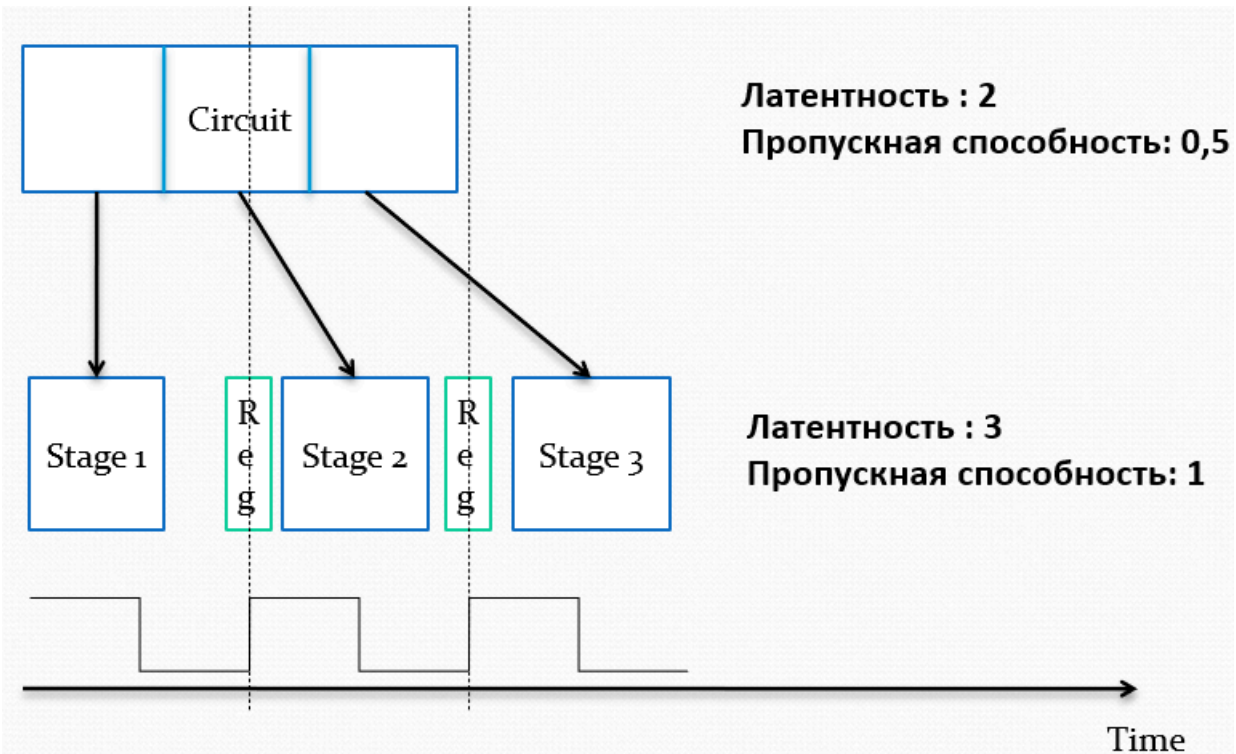


Рис. 3. **CDC 6600 (1967 г.)**

Конвейер (продолжение)



Конвейер инструкций

- Стадии:
  - выборка инструкции
  - выборка данных из регистров и декодирование инструкции
  - исполнение инструкции (вычисление адреса)
  - обращение к памяти (загрузка и сохранение)
  - запись данных в регистры

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Instruction i	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Что препятствует эффективному исполнению инструкций на конвейере?



- зависимости между инструкциями
- наличие зависимостей приводит к возникновению конфликтов (hazard)

КОНФЛИКТЫ (HAZARDS) И ИХ РАЗРЕШЕНИЕ

Конфликт



- конфликт - это некоторая ситуация, при которой невозможно запустить следующую инструкцию непосредственно за предыдущей, не нарушив критерия сохранения корректности программы
- запуск инструкции происходит спустя n тактов
- не атрибут программы или алгоритма, это следствие параллельного исполнения зависимых инструкций на процессоре

Критерии сохранения корректности программы



- сохранение потока данных
  - поток значений передаваемых между инструкциями производителями и потребителями должен быть сохранен
  - инструкции ветвления делают поток данных динамическим
- сохранение поведения исключений
  - порядок, количество и тип генерируемых исключений должен сохраняться

Типы конфликтов



- структурные конфликты
  - разным инструкциям требуется доступ к одному ресурсу процессора
- конфликты по данным
  - следствие наличия зависимостей по данным и именам
- конфликты по управлению
  - следствие наличия зависимости по управлению

Конфликты по данным



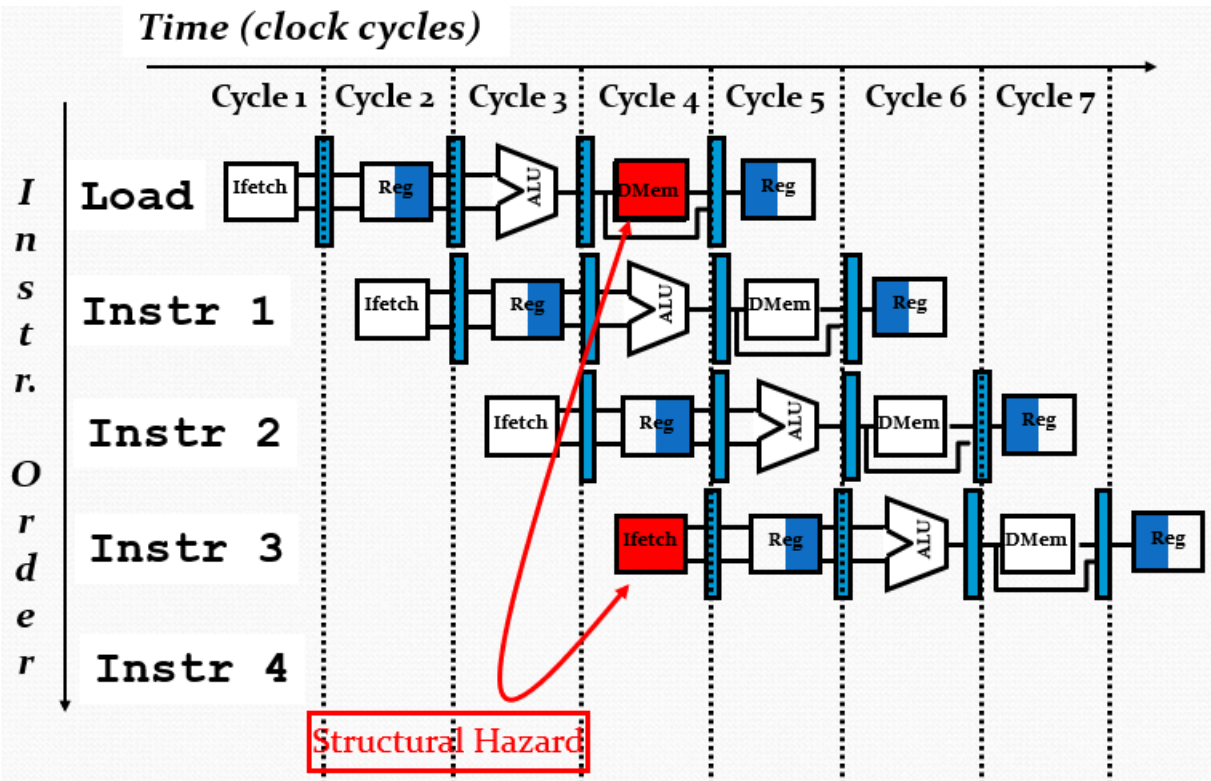
- **RAW (read after write).** Чтение после записи
  - i. R2 <- R1 + R3
  - j. R4 <- R2 + R3
  - возможен для скалярных и суперскалярных конвейерных процессоров
- **WAR (write after read).** Запись после чтения
  - i. r1 <- r2 + r3
  - j. r3 <- r4 + r5
  - возможен для суперскалярных процессоров
- **WAW (write after write).** Запись после записи
  - i. r2 <- r1 + r3
  - j. r2 <- r4 + r7
  - возможен для суперскалярных процессоров

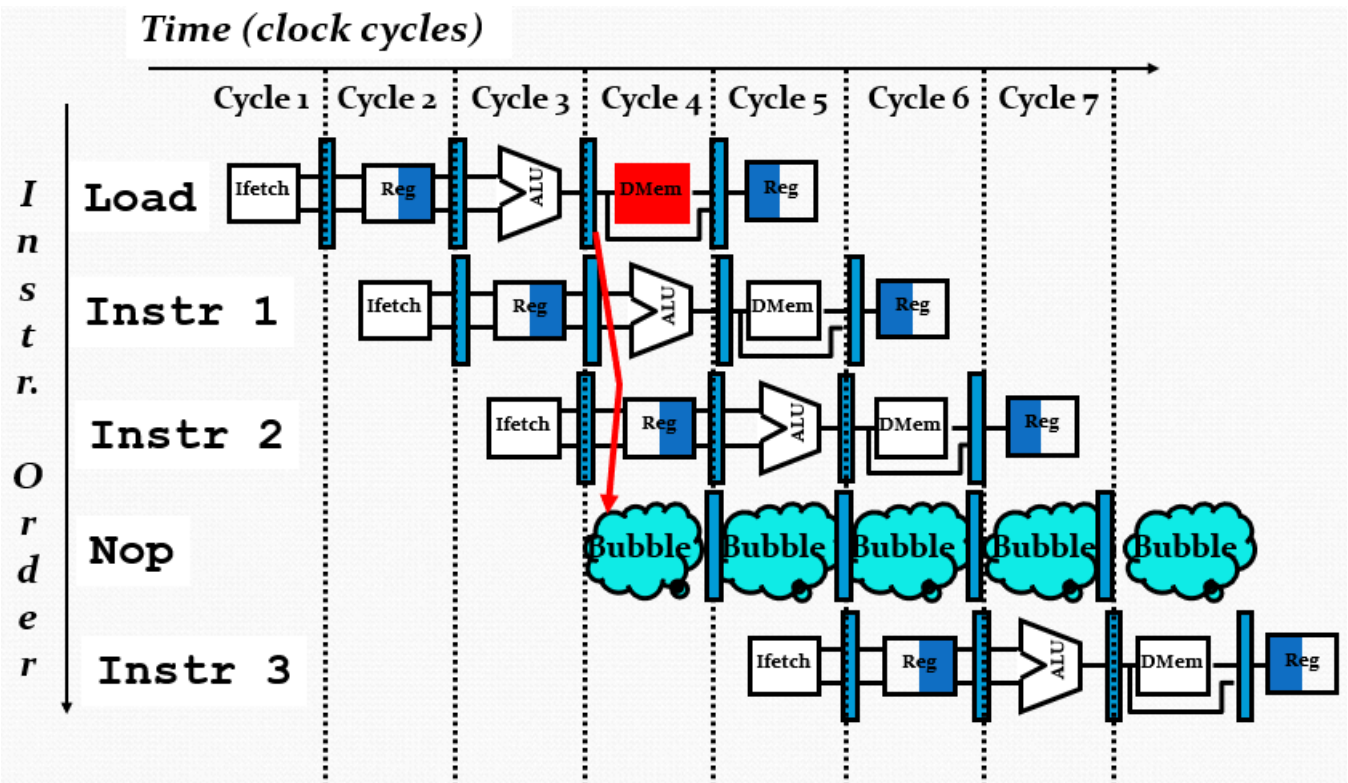
Способы разрешения конфликтов



- программные
  - обнаружение конфликтов в процессе компиляции и добавление инструкций nop
  - статическое планирование конвейера
- аппаратные
  - добавление устройства обнаружения конфликтов и останова конвейера
  - добавление специфичных аппаратных решений

Структурные конфликты и механизмы их устранения

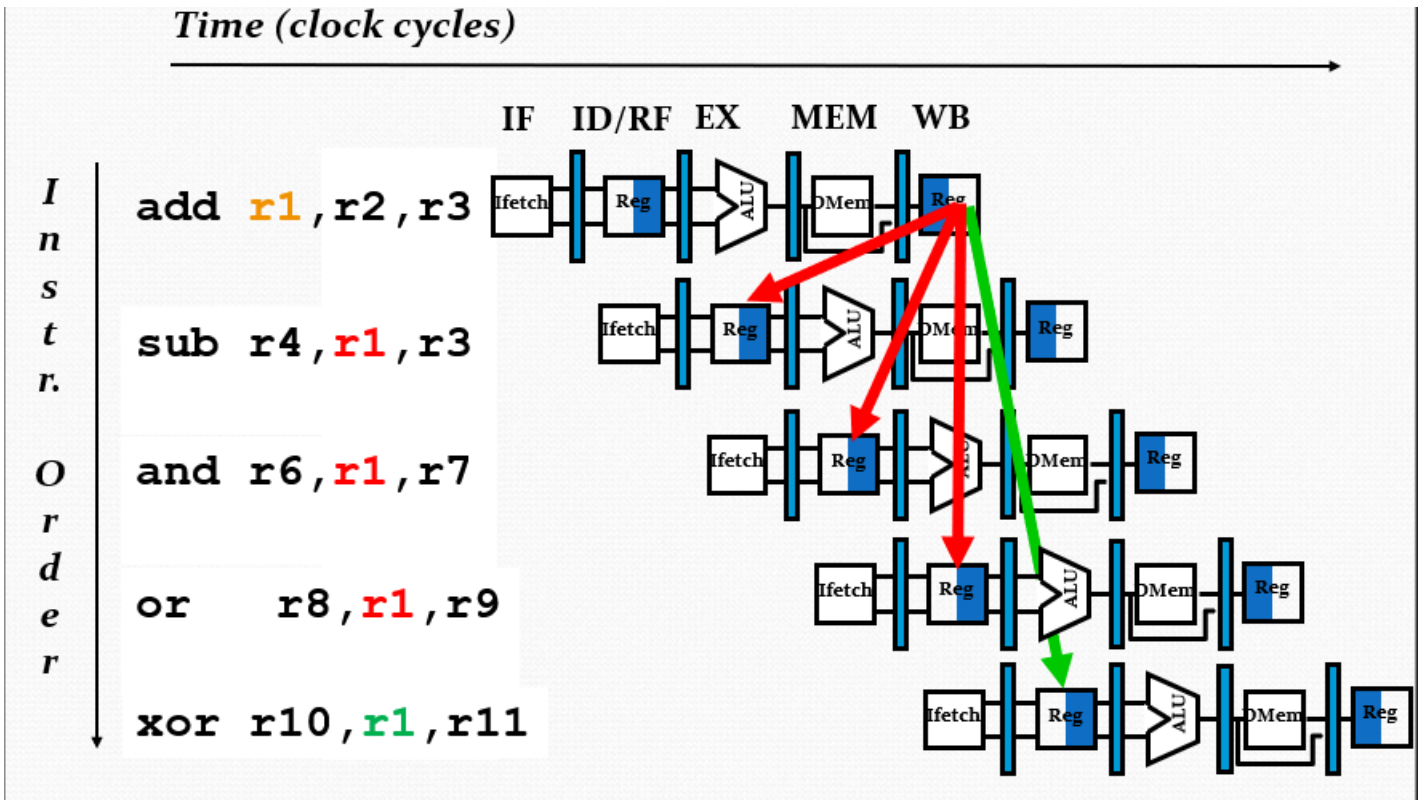


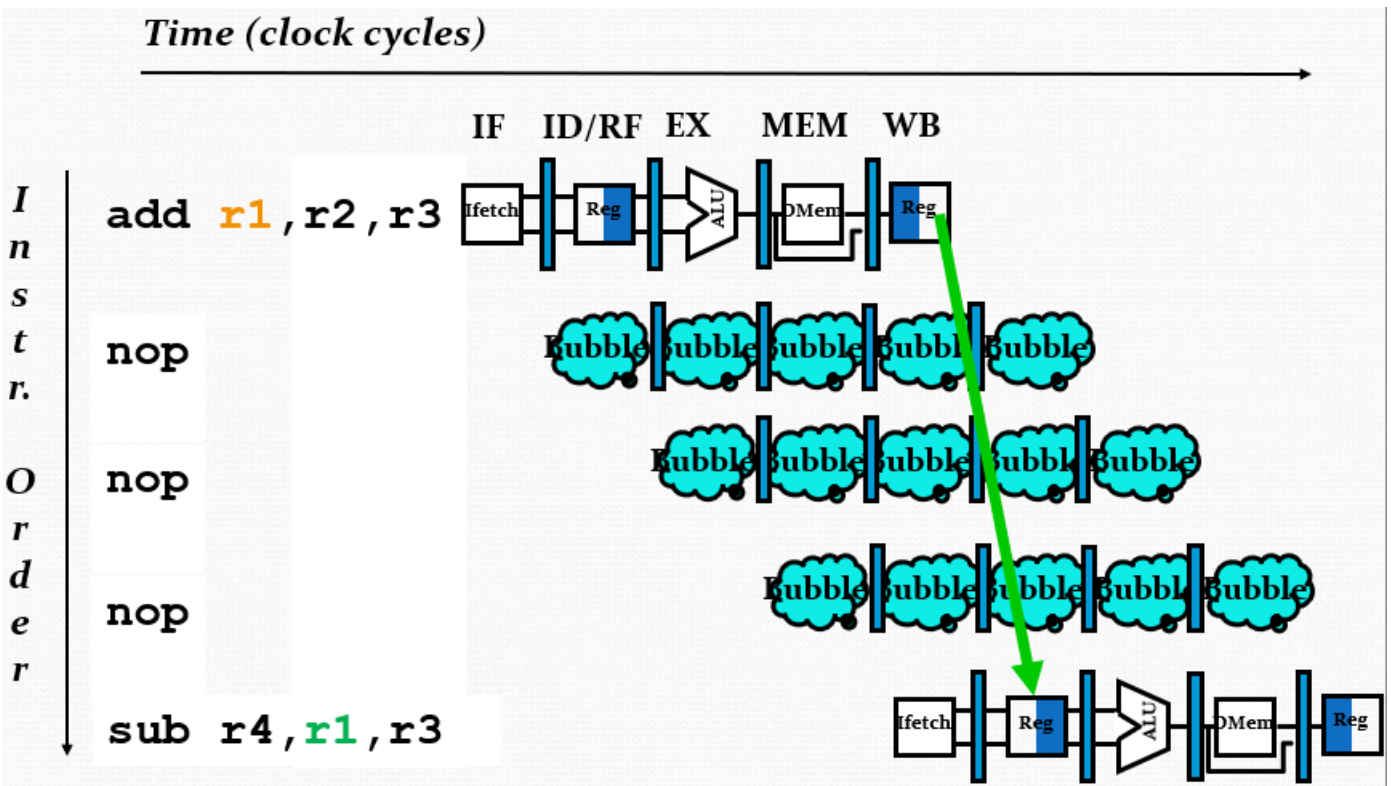

Time (clock cycles)
Cycle 1
Cycle 2
Cycle 3
Cycle 4
Cycle 5
Cycle 6
Cycle 7
Instr. Order
Load
Instr 1
Instr 2
Nop
Instr 3
Ifetch
Reg
ALU
DMem
Reg
Bubble



- второй способ устранения:
  - второй канал к памяти
  - разделение общего кэша на кэш данных и кэш инструкций

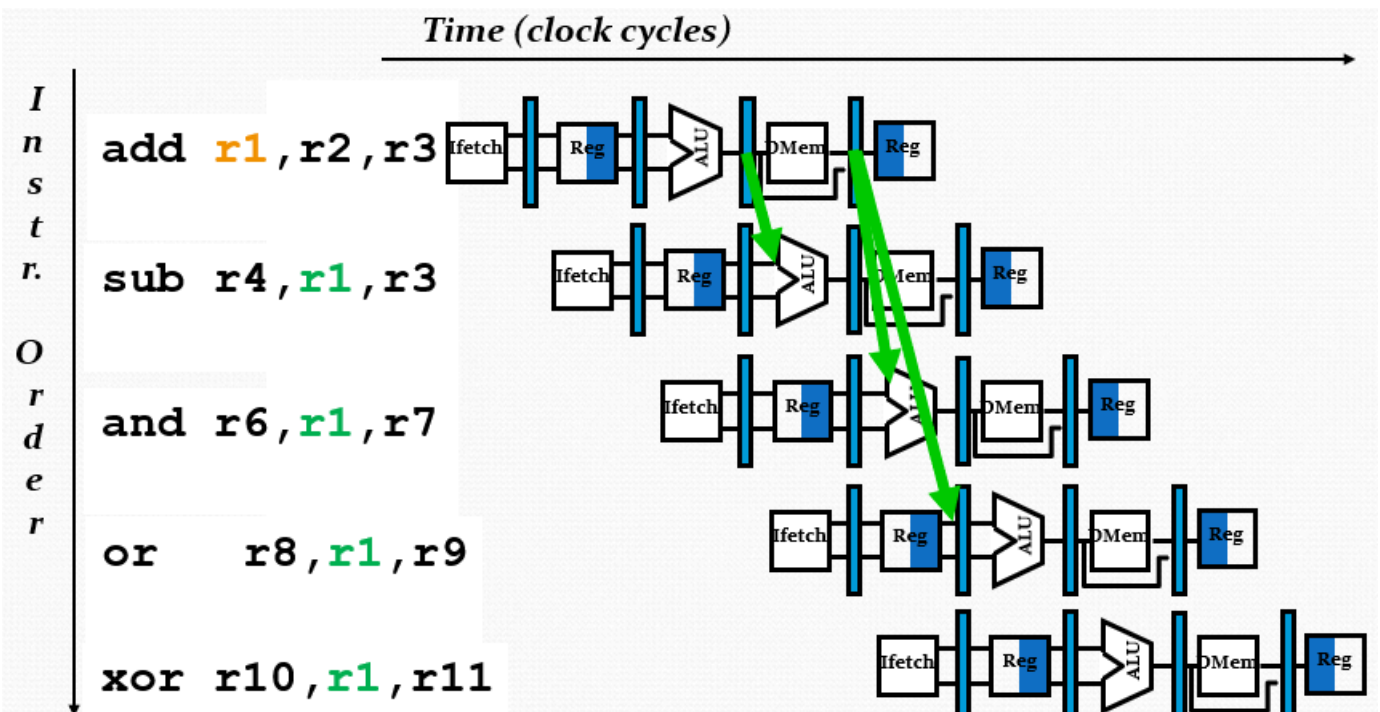
Конфликт RAW

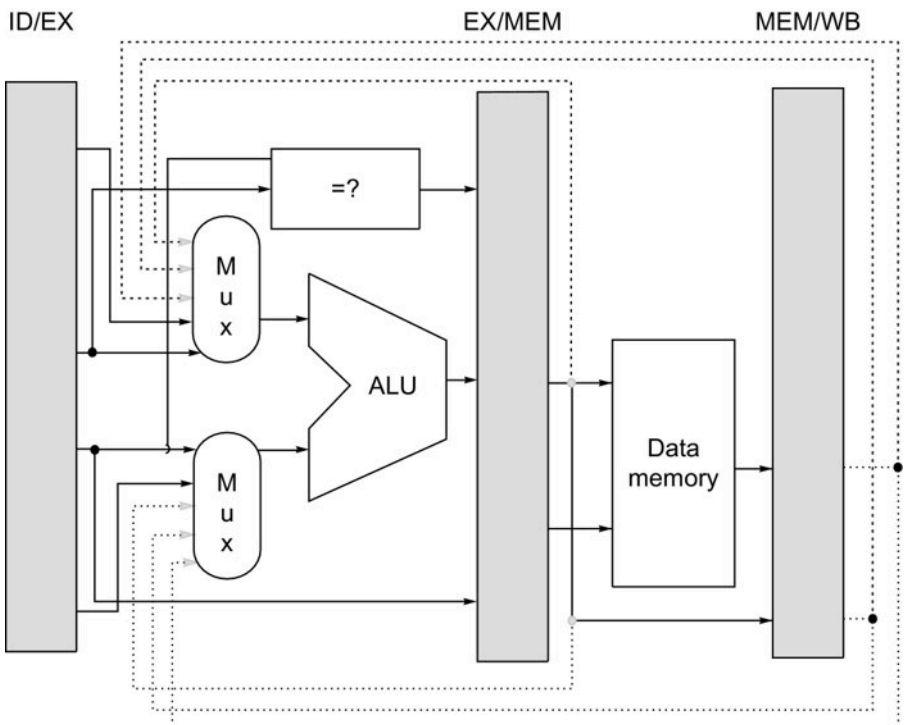



Time (clock cycles)
IF
ID/RF
EX
MEM
WB
Instr. Order
add r1,r2,r3
nop
nop
nop
sub r4,r1,r3
Ifetch
Reg
ALU
DMem
Reg
Bubble

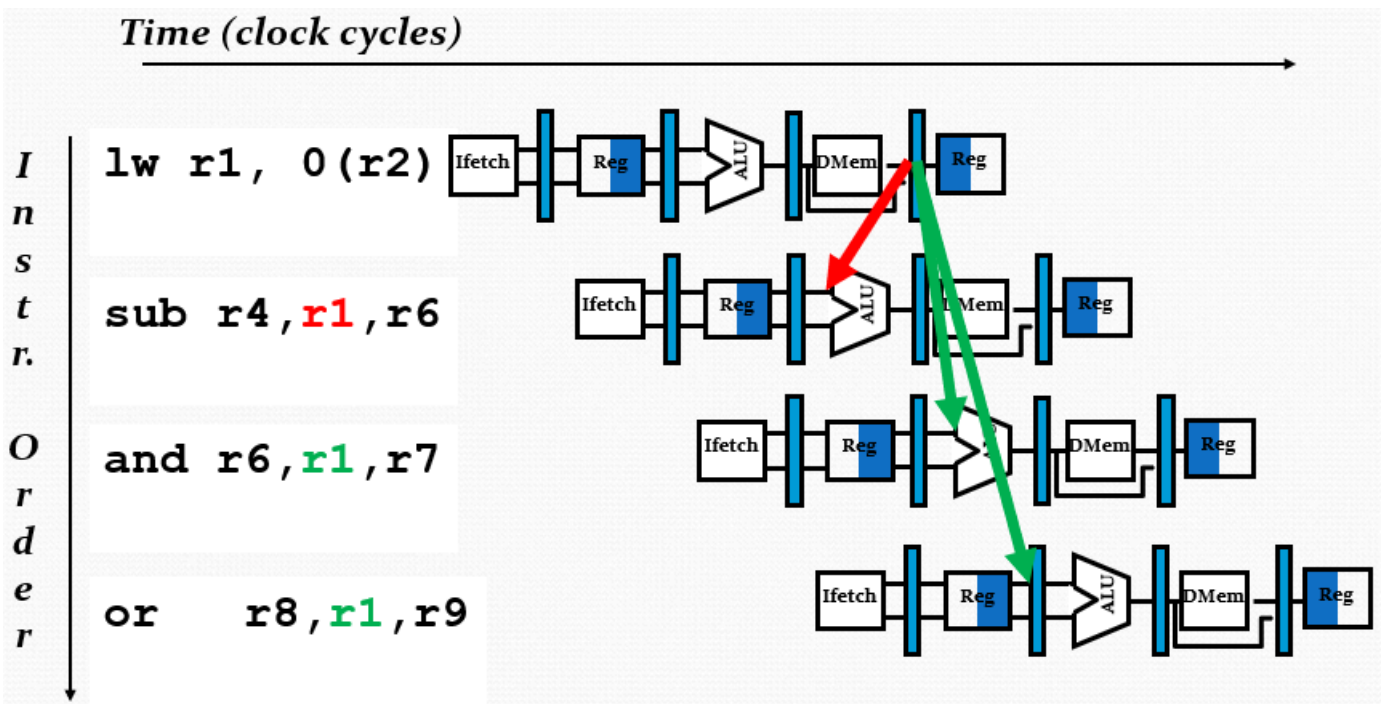
Метод bypass

- сводится к возможности передачи данных между стадиями конвейера напрямую
- вход стадии может быть соединен с выходом любой последующей стадии




ID/EX
EX/MEM
MEM/WB
=?
M
u
x
ALU
M
u
x
Data
memory

Ограничения bypass (проблема загрузки)

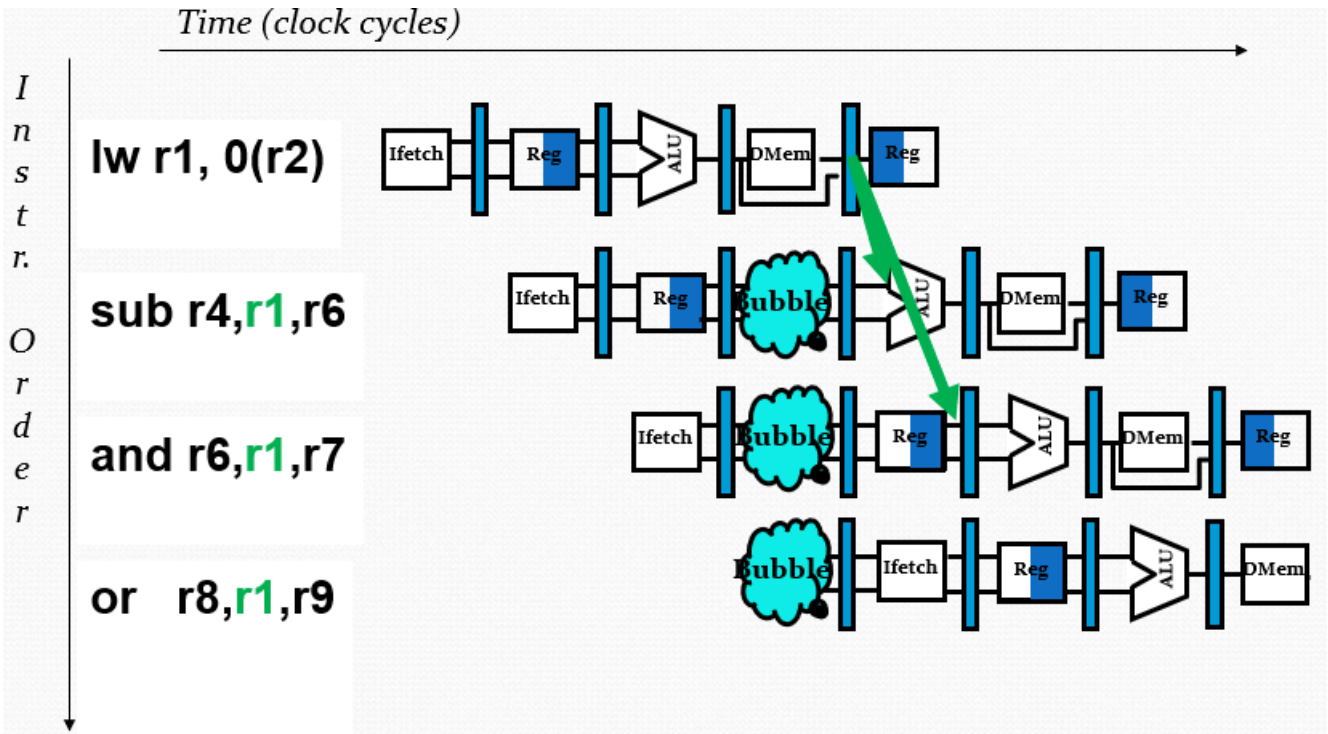


Ограничения bypass (проблема загрузки)

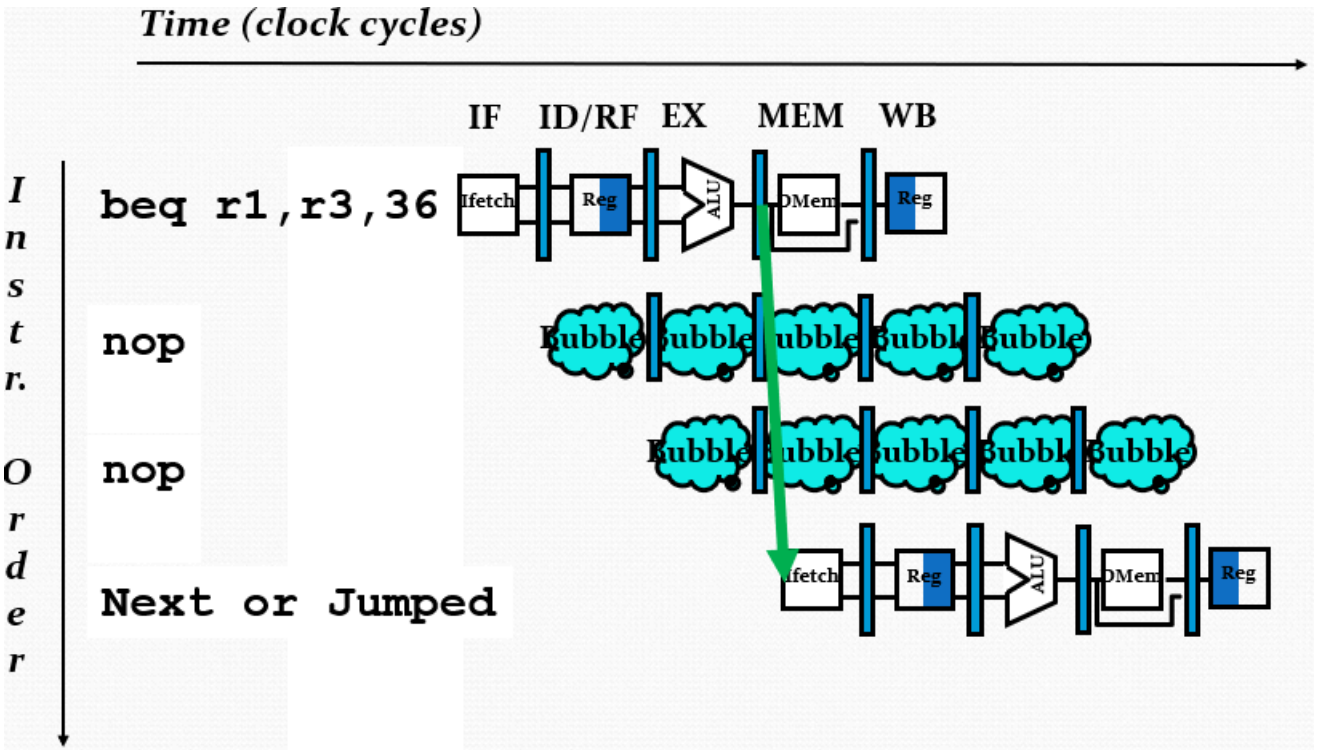


- данные могут быть переданы на другую стадию, только в том случае, если они реально существуют
- необходимым и достаточным условием работы bypass механизма является наличие требуемых данных хотя бы на одной стадии конвейера к тому моменту, когда они действительно потребуются

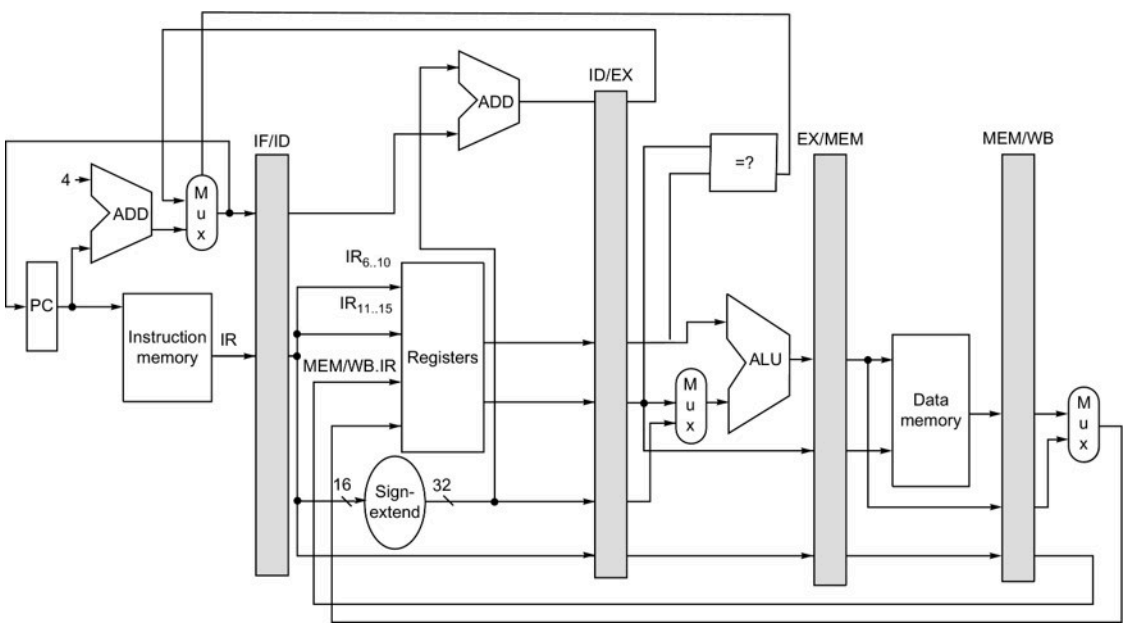
Решение проблемы загрузки остановкой конвейера



Конфликт управления



Как уменьшить количество тактов простоя?

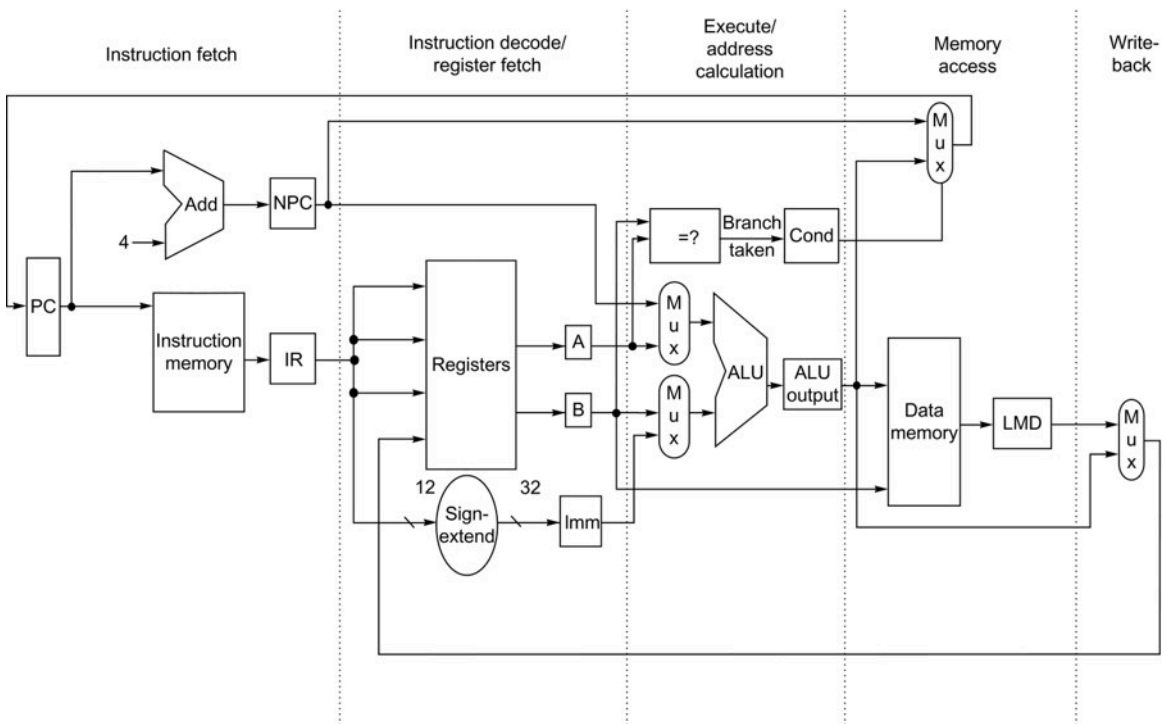


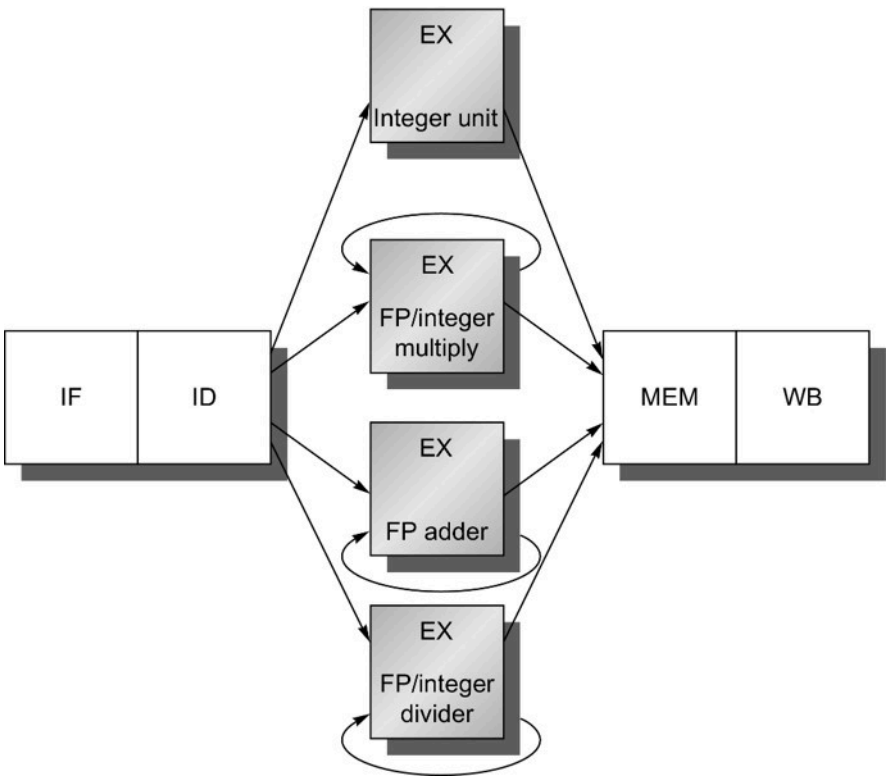
Неточное исключение



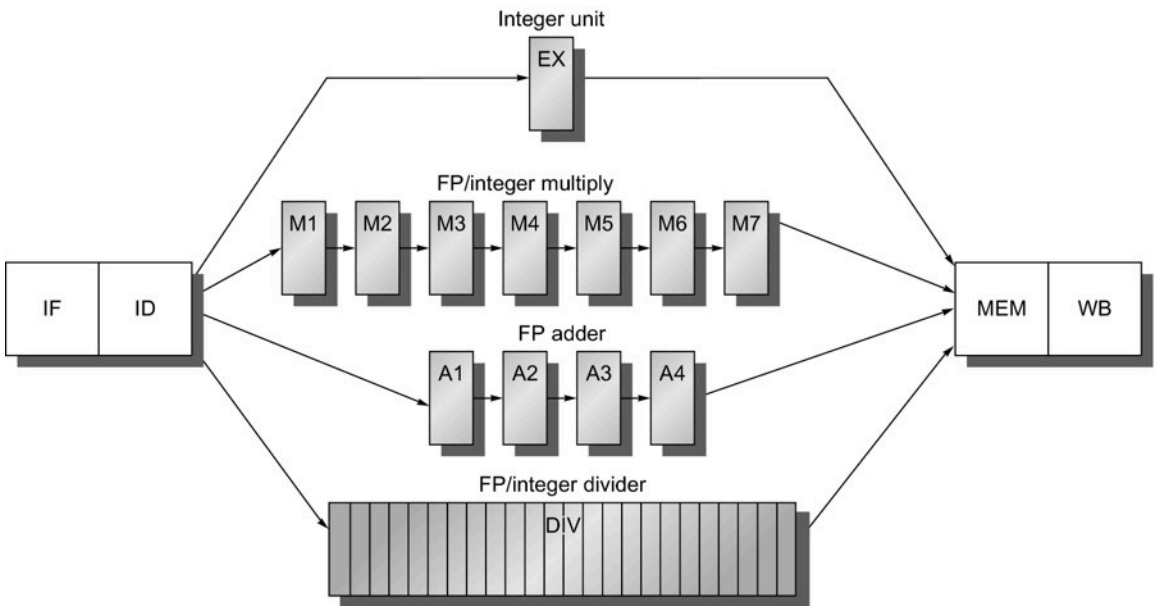
- это исключение, которое сгенерировано не в своем регистровом контексте
- возникает при внеочередном исполнении инструкций
- причины:
  - на конвейере уже выполнилась инструкция, которая следует за инструкцией, сгенерировавшей исключение
  - на конвейере еще выполняется инструкция, которая предшествует инструкции, сгенерировавшей исключение

Практикум RISC-V




EX
Integer unit
EX
FP/integer multiply
EX
FP adder
EX
FP/integer divider
IF
ID
MEM
WB

Практикум: проблема разной латентности инструкций



АЛГОРИТМ ТОМАСУЛО

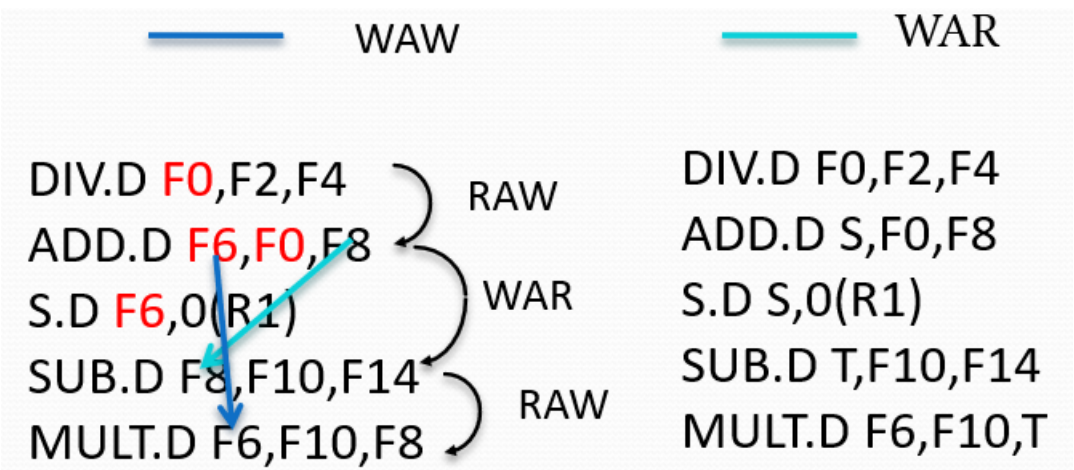
Алгоритм Томасуло (80-е годы, IBM)

- Реализован в IBM 360/91 FPU
- Алгоритм динамического планирования инструкций, который позволяет исполнять инструкции в порядке отличном от программного
- при одновременном исполнении двух и более инструкций позволяет разрешить RAW, и устранить WAR и WAW конфликты



Применение переименования регистров для устранения WAR и WAW конфликтов

- переименование регистров позволяет устранить WAR и WAW

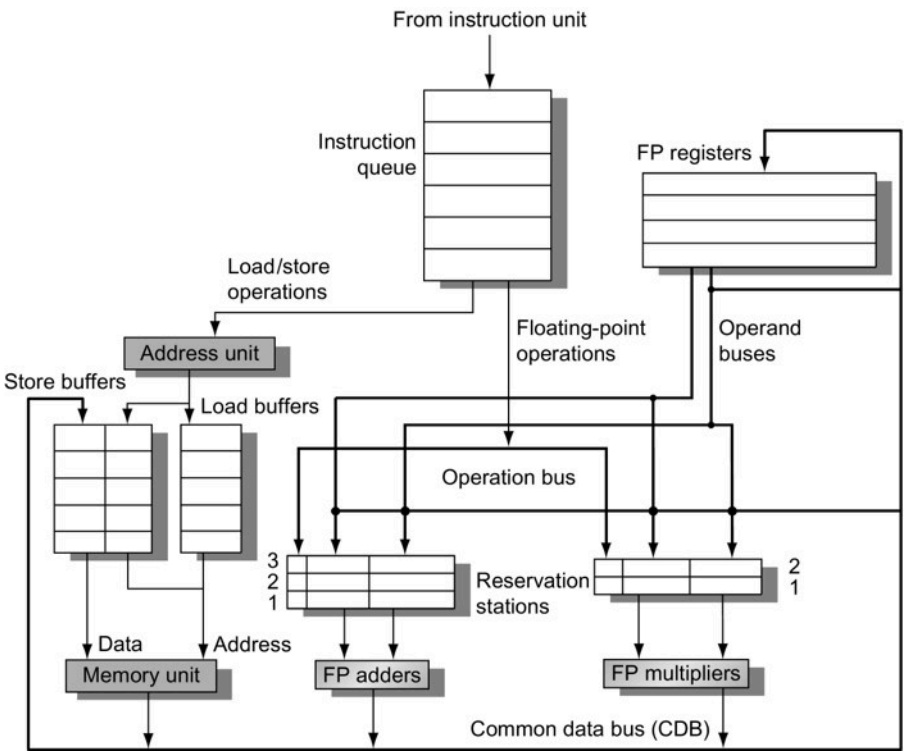


Алгоритм Томасуло



- разрешение конфликтов RAW происходит за счет запуска инструкции, только когда готовы ее операнды
- устранение WAR и WAW конфликтов происходит за счет переименования регистров с использованием станций резервирования (Reserve Station)
- RS используются для хранения операндов инструкции и воссоздания графа зависимостей по данным между инструкциями, которые находятся в исполнении

Схема процессора для реализации алгоритма Томасуло



Состав процессора



- Очередь планирования (Instruction queue)
- Регистровый файл (FP Registers) [R]
- Станции резервирования (Reserve Station) [RS]
- Простое вещественное устройство (FP adder)
- Сложное вещественное устройство (FP Multiplier)
- Общая шина данных (Common Data Bus) [CDB]
- Устройство вычисления адреса (Address Unit)
- Буфер загрузки (Load buffer)
- Буфер сохранения (Store buffer)
- Устройство работы с памятью (Memory Unit)

Этапы исполнения инструкции



Классический конвейер

- выборка инструкции
- выборка операндов
- исполнение
- сохранение результата

Алгоритм Томасуло

- выборка инструкции
- планирование инструкции
- ожидание готовности операндов
- исполнение
- сохранение результата

Устройство RS



- состоит из двух дескрипторов операндов
- дескриптор операнда содержит значение операнда или ссылку, если значение операнда на момент планирования еще не вычислено
- ссылка - это номер RS, который содержит инструкцию вычисляющую данный операнд

Планирование инструкций



- выборка с вершины
  - выборка происходит по 1 инструкции за такт
  - выборка осуществляется в программном порядке, так как очередь FIFO
- декодирование
- назначение на исполнительное устройство
  - если все RS устройства заняты, то инструкция возвращается в IQ и ожидает освобождения RS
- выборка операндов
  - если операнды вычислены, то они выбираются из R, если нет то в дескрипторах RS устанавливаются ссылки на другие RS

Ожидание готовности операндов, исполнение, сохранение результатов



- инструкция ожидает в RS до тех пор, пока не будут вычислены все ее операнды и записаны в соответствующие дескрипторы
- передача вычисленного операнда происходит по CDB вместе с номером RS, который его содержал
- каждая RS слушает CDB и сравнивает значение номер передаваемого по ней RS с ожидаемым. Если номера совпадают то она забирает значение операнда с CDB
- передаваемые по CDB данные сохраняются в R
- если все операнды находятся в RS, то инструкция отправляется на исполнение

Обработка инструкций загрузки и сохранения



- Инструкции загрузки:
  - вычисление адреса
  - выполнение загрузки по адресу
- Инструкции сохранения:
  - вычисление адреса
  - ожидание готовности операнда
  - выполнение сохранения по адресу
- Буфер загрузки: поле адреса
- Буфер сохранения: поле адреса, RS для операнда

Порядок исполнения инструкций загрузки и сохранения



- определяется наличием зависимостей между инструкциями по ячейкам памяти
- адреса ячеек вычисляются на первом этапе
- инструкции загрузки ожидает завершения всех предшествующих инструкций сохранения по данному адресу
- инструкции сохранения ожидает завершения всех предшествующих инструкций загрузки и сохранения по данному адресу

Файл переименования



- находится в R
- содержит имена переименованных регистров
- состоит из двух полей:

Регистр	**Ссылка**

- если значение находиться в регистре, ссылка равна нулю, если нет то ссылка указывает на RS, которая его вычислит

ПРИМЕР РАБОТЫ И ТАБЛИЦА СОСТОЯНИЙ

Пример



1. L.D F6,34(R2)
2. L.D F2,45(R3)
3. MUL.D F0,F2,F4
4. SUB.D F8,F2,F6
5. DIV.D F10,F0,F6
6. ADD.D F6,F8,F2

- Допущения
- Латентность загрузки/сохранения: 1
- Латентность сложения/вычитания: 2
- Латентность умножения: 5
- Латентность деления: 10
- RS для загрузки/сохранения: 2
- RS для простых арифм. опер.: 3
- RS для сложных арифм. опер.: 2
- Планирование инструкций происходит отдельно от процесса исполнения.

Таблица состояния RS

RS	RS tag	Busy	Op	Vi	Vj	Qi	Qj	A
Load1	01	yes	Load					34+Regs[R2]
Load2	02	yes	Load					45+Regs[R3]
Add1	03	yes	Sub			02	01	
Add2	04							
Add3	05	no						
Mul1	06	yes	Mul		Regs[F4]	02	0	
Mul2	07	yes	Div			06	01	

Register	F0	F2	F4	F6	F8	F10
RS tag	06	02		01	03	07

Исполнение инструкций по плану



1. L.D F6,34(R2)
2. L.D F2,45(R3)
3. MUL.D F0,F2,F4
4. SUB.D F8,F2,F6
5. DIV.D F10,F0,F6
6. ADD.D F6,F8,F2

RS	RS tag	Busy	Op	Vi	Vj	Qi	Qj	A
Load1	01							
Load2	02							
Add1	03	yes	Sub					
Add2	04	yes	Add			03		
Add3	05	no						
Mul1	06	yes	Mul		F4			
Mul2	07	yes	Div					

Register	F0	F2	F4	F6	F8	F10
RS tag	06			04	03	07

Более полное описание алгоритма

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	`if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rt].Qi else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Q ← r;`
Load or store	Buffer r empty	`if (RegisterStat[rs].Qi≠0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;`
Load only		`RegisterStat[rt].Qi ← r;`
Store only		`if (RegisterStat[rt].Qi≠0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};`
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/storestep 1	RS[r].Qj = 0 & r is head of load-store queue	`RS[r].A ← RS[r].Vj + RS[r].A;`
Load step 2	Load step 1 complete	Read from `Mem[RS[r].A]`
Write result FP operation or load	Execution complete at r & CDB available	`∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result;RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result;RS[x].Qk ← 0}); RS[r].Busy ← no;`
Store	Execution complete at r & RS[r].Qk = 0	`Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;`

Достоинства и недостатки алгоритма



- Достоинства:
  - повышение пропускной способности
  - уменьшение времени простоя процессора
- Недостатки:
  - большие аппаратные затраты на реализацию дополнительных устройств



СТАТИЧЕСКИЕ И ДИНАМИЧЕСКИЕ СУПЕРСКАЛЯРНЫЕ ПРОЦЕССОРЫ

Суперскалярный процессор



- процессор, который способен закачивать или оставлять 2 и более команды за такт
- скалярный конвейерный процессор исполняет более одной инструкции, но завершает всегда по одной

Типы суперскалярных процессоров



- Статический
  - выполняет инструкции по порядку (in-order)
- Динамический
  - выполняет инструкции вне порядка (out of order), кроме инструкций условного перехода
- Спекулятивный
  - выполняет все инструкции вне порядка (out of order)
  - спекулирует над инструкциями условного перехода и инструкциями загрузки/сохранения

Типы процессоров

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the Cortex-A53
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Статический суперскалярный процессор



- Похож с классическим конвейерным скалярным процессором
- Все инструкции выполняются в рамках одного конвейера
- Появилась дополнительная стадия планирования и группировки инструкций
- Целочисленные и вещественные части конвейера выравниваются по длине с использованием пустых стадий

Стадия планирование и группировки инструкций



- Проверяет группу из n инструкций на наличие конфликтов внутри группы и с инструкциями, которые уже находятся на исполнении
  - если внутри группы инструкций обнаружен конфликт
    - на исполнение отправляются инструкции, которые предшествуют первой конфликтной инструкции в программном порядке, остальные инструкции ждут разрешения конфликта
  - также стадия отвечает за распределение инструкций между устройствами

Достоинства и недостатки



- Достоинства:
  - относительная аппаратная простота
  - высокая пиковая производительность
  - возможно достижение высокой рабочей частоты
- Недостатки:
  - требуется перекомпиляция кода для достижения высокого реального быстродействия

Модели статических суперскалярных процессоров

- Sun
  - Ultra Sparc III (2001 г., 600-900 МГц)
  - Ultra Sparc IV (2004 г., 4 до 1,5 ГГц)
    - первый многоядерный SPARC



- POWER 6 (2007 г., 3,5-5(6) ГГц)



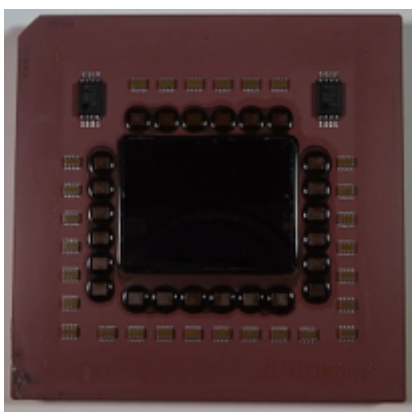


Схема процессора Sun UltraSPARC III

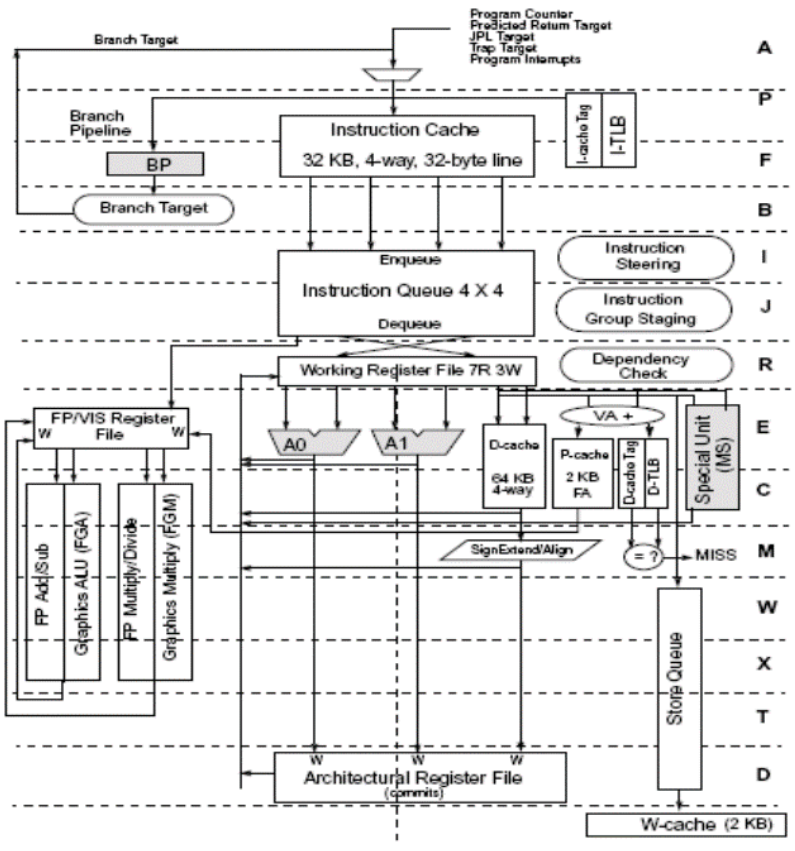
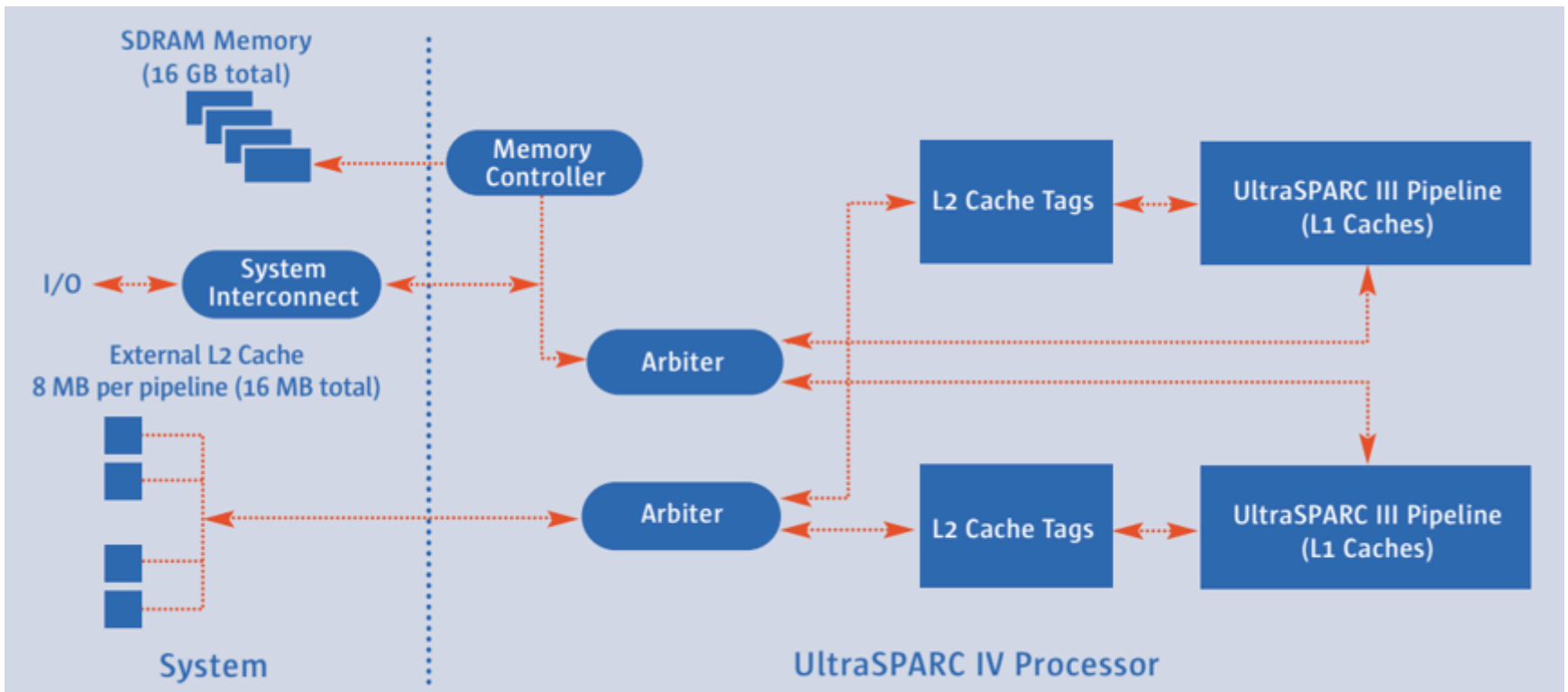
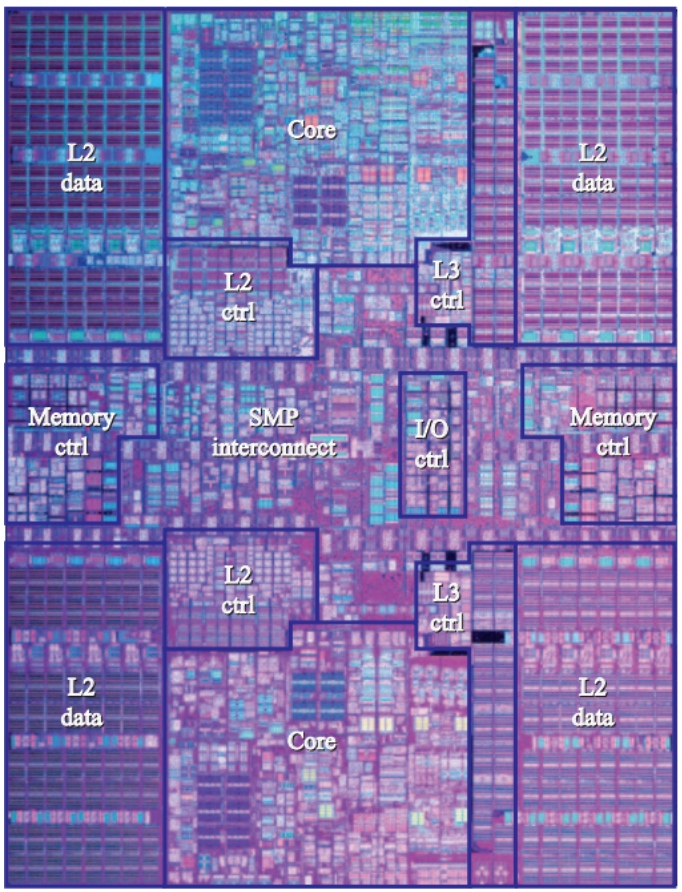


FIGURE 4-1 Instruction Pipeline Diagram

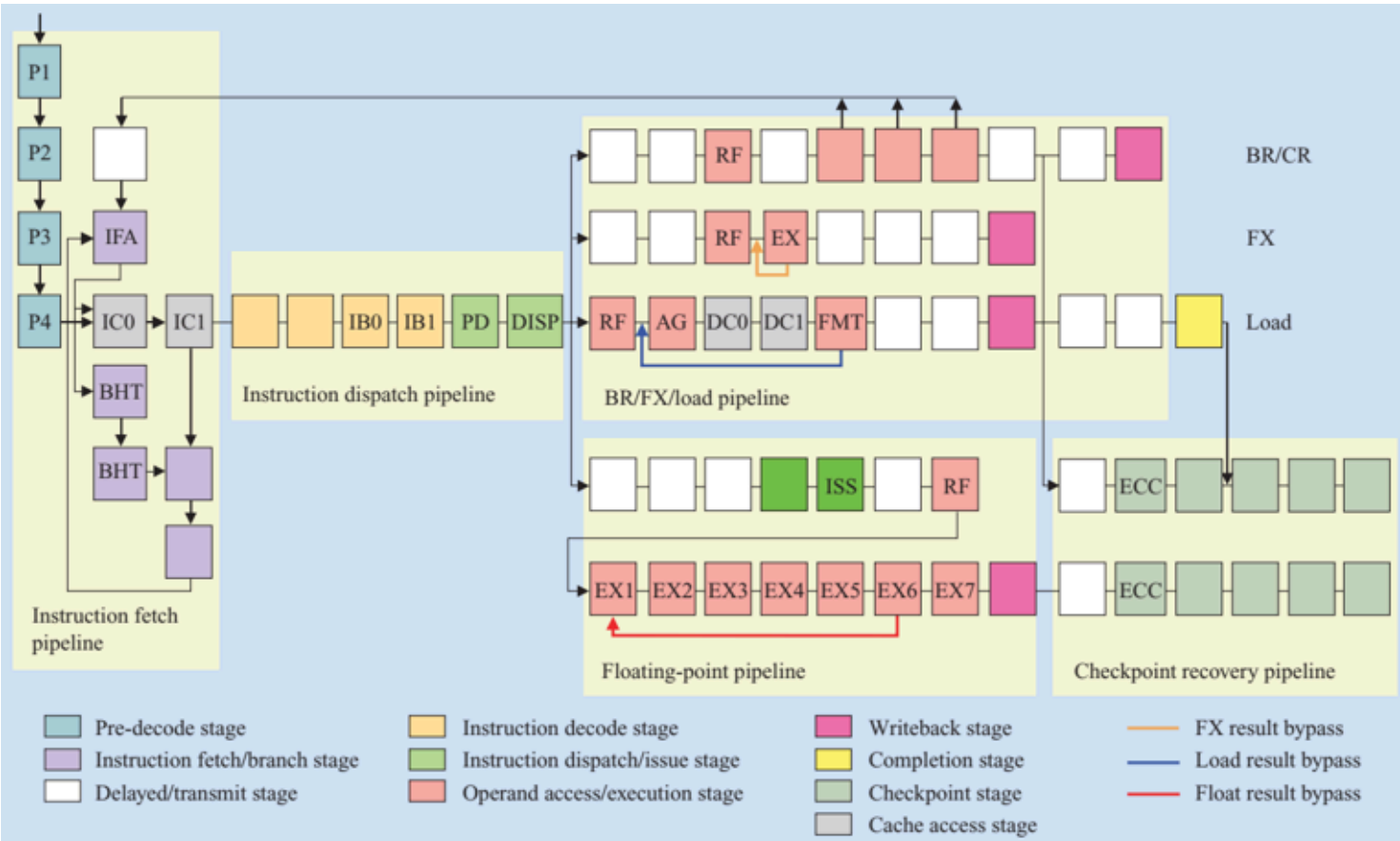
Схема процессора Sun UltraSPARC IV



IBM Power 6


L2
data
Core
L2
data
L2
ctrl
L3
ctrl
Memory
ctrl
SMP
interconnect
I/O
ctrl
Memory
ctrl
L2
ctrl
L3
ctrl
L2
data
Core
L2
data

Конвейер IBM Power 6



AG: address generation; BHT: branch table access and predict; BR: branch; DC: data-cache access; DISP: dispatching; ECC: error-correction code; EX: execute; FMT: formatting; IB: instruction buffer; IC0/IC1: instruction-cache access; IFA: instruction fetch address; ISS: issue; P1-P4: pre-decode; PD: post-decode; RF: register file access

Динамический суперскалярный процессор



- Для планирования использует алгоритм Томасуло.
- Присутствуют стадии планирования, которая может планировать более 2 инструкций за такт
- Инструкции после инструкции условного перехода планируются, но не исполняются, до исполнения инструкции условного перехода

SUN ULTRASPARC III И IBM POWER 6

Sun Ultra SPARC III



- Является статическим суперскалярным процессором
- Поколение - SPARC Version 9 (64-битное расширение)
- Максимальное устойчивое количество инструкций обрабатываемых за такт - 4 (ограничивается скоростью выборки)
- Максимальное количество инструкций исполняемых и оставляемых за такт - 6
- Существует целое семейство процессоров SUN построенных по аналогичной микроархитектуре: UltraSPARC III (Cu,i), UltraSPARC IV/ IV+
- Длина конвейера - 14 стадий

Sun Ultra SPARC III

Table 1. UltraSPARC-III pipeline and physical data.

Pipeline feature	Parameter
Instruction issue	4 integer
	2 floating-point
	2 graphics
Level-one (L1) caches	Data: 64-Kbyte,4-way
	Instruction: 32-Kbyte,4-way
	Prefetch: 2-Kbyte,4-way
	Write: 2-Kbyte,4-way
Level-two (L2) cache	Unified (data and instruction):
	4- and 8-Mbyte, 1-way
	On-chip tags
	Off-chip data
Physical feature	**Parameter**
Process	0.25-micron CMOS, 6 metal layers
Clock	600+ MHz
Die size	360 mm^2
Power	760 watts @1.8 volts
Transistor count	RAM: 12 million
	Logic: 4 million
Package	1,200-pin LGA

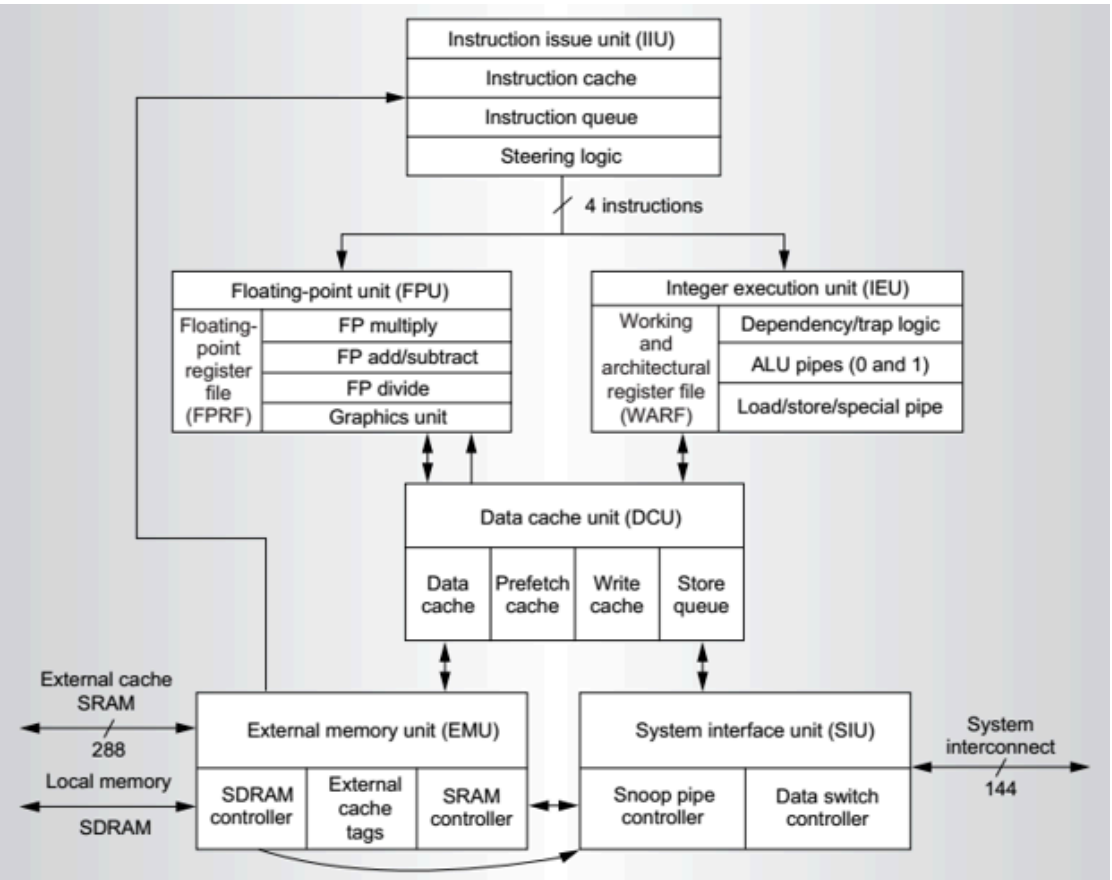


Sun UltraSPARC III. С точки зрения разработчиков



- Особенности:
  - обратная совместимость
  - заявленное ускорение без (!) перекомпиляции кода: до 90% (!!!)
    - масштабируемость архитектуры
    - Instruction Level Parallelism (ILP) - единица измерения, показывающая среднее число инструкций в программе, которое может быть выполнено в единицу времени
    - пропускная способность шины
    - латентность инструкций

Состав исполняемых устройств



Instruction issue unit



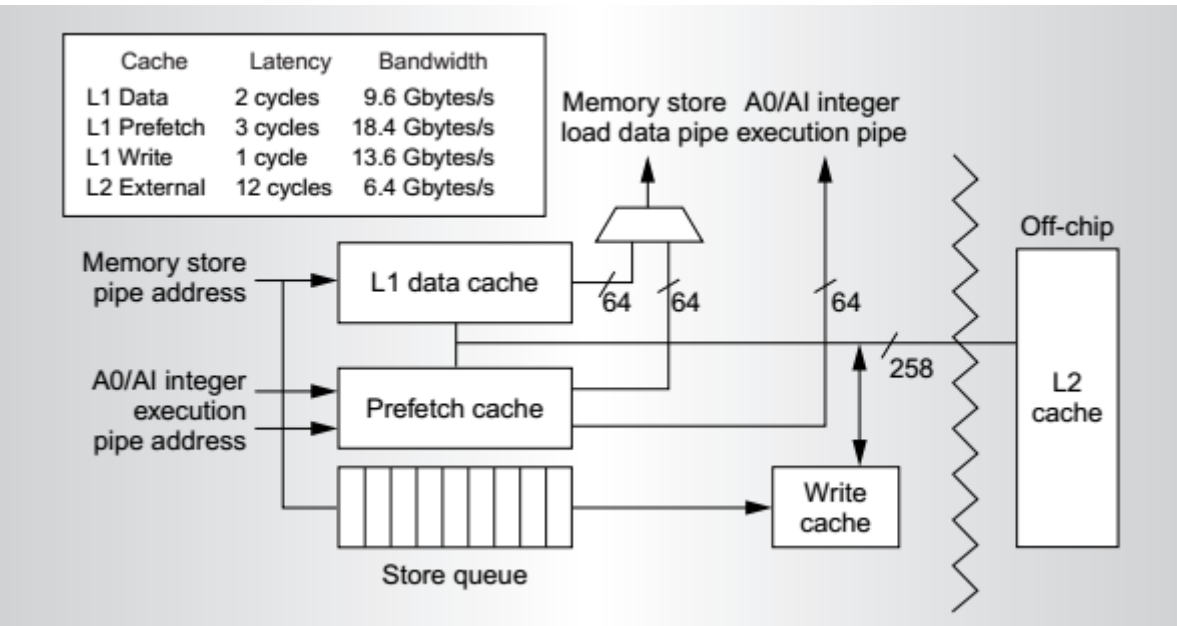
- Выборка инструкций из ОЗУ
- Блок предсказания перехода
- 32 к 4-ассоциативный кэш инструкций
- Буфер трансляции адресов инструкций (16 кБ)

Состав исполняемых устройств



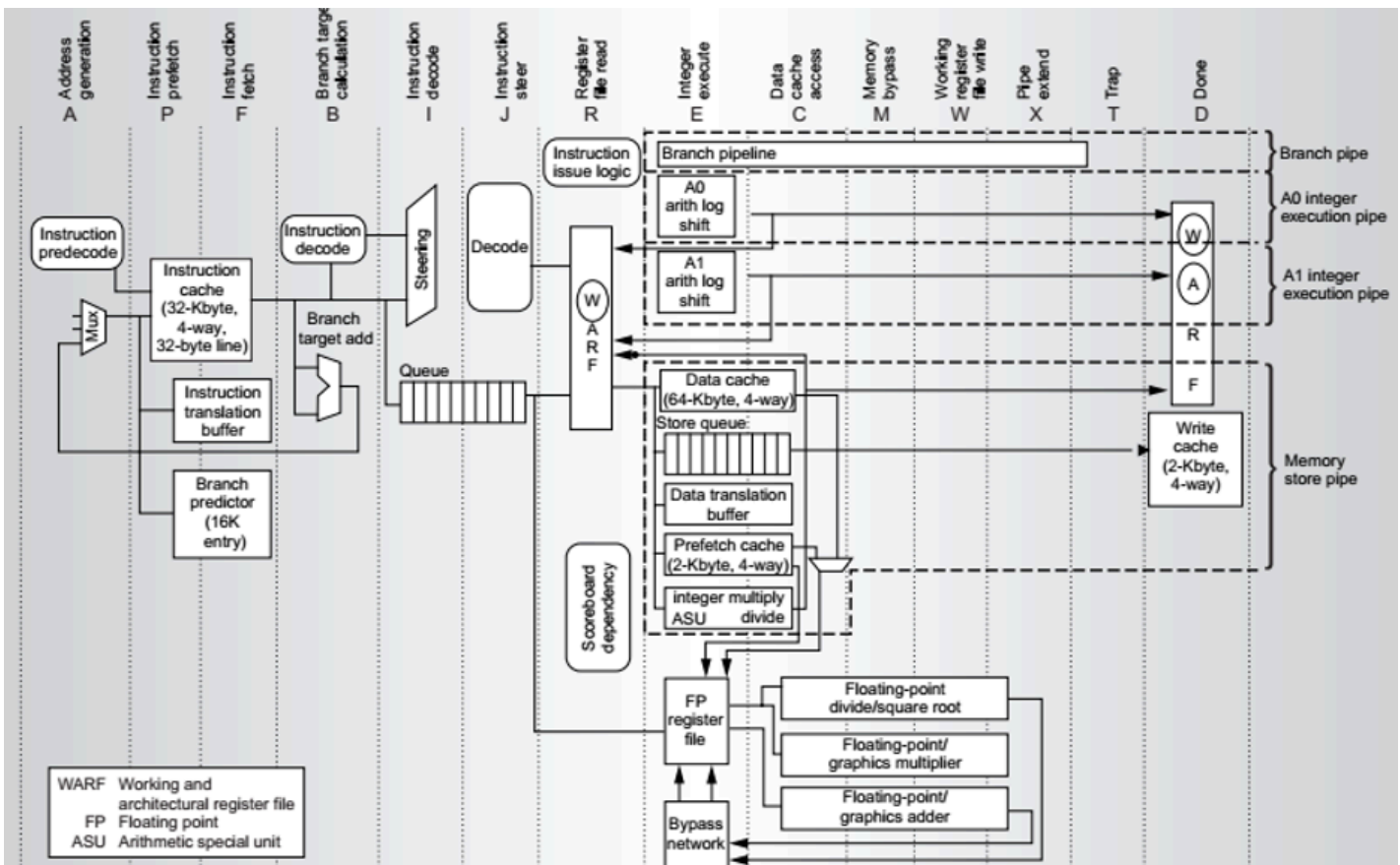
- 2 целочисленных АЛУ
- 1 устройство загрузки/сохранения
- 1 устройство исполнения специальных команд
- 1 простое вещественное устройство (сложение/ вычитание) / простое векторное устройство
- 1 сложное вещественное устройство (умножение/деление) / сложное векторное устройство

Кэш данных



- Кэш данных - 64-Кбайт, 4-ассоциативный, 32 кэш строки
- Prefetch-кэш - 2-Кбайт, 4-ассоциативный, 64 байта кэш строки;
- Кэш для записи - 2-Кбайт, 4-ассоциативный, 64 байта кэш строки;

Пример Sun UltraSPARC III



Описание стадий конвейера



- Стадия конвейера (S) отвечает за вычисление адреса следующей инструкции. Может быть вычислено на основе счетчика команд, вычисленной точки перехода, и адреса из стека возврата
- Стадия конвейера (P) отвечает за начало выборки команд из I-кэш инструкций. Также вычисленный адрес отдается блоку предсказания ветвления, который дает информацию о вероятной точке перехода
- Стадия F закачивает выборку инструкций из I-кэш. Выбирается 4 инструкции
- Четвертая стадия T проверяет, правильно ли был предсказан адрес для следующей инструкции, если нет, то повторяем выборку
- Стадия I помещает инструкции в очередь 4 на 4
- На стадии J инструкции выбираются из очереди. Инструкции могут находиться в очереди произвольное количество тактов, до тех пор, пока все предшествующие инструкции не будут выбраны

Стадия R



- На стадии R выбранная четверка инструкций осуществляет доступ к требуемым регистровым данным. Для целочисленных команд может быть одновременно выбрано 7 операндов: 4 для 2-х арифметических инструкций, 2 для инструкции ветвления, и 1 для инструкции сохранения
- Одновременно с выборкой данных на стадии R вычисляются зависимости между инструкциями, уже находящимися на исполнении, и всеми инструкциями которые находятся на стадии R. На исполнение отправляются только те инструкции, перед которыми в программном порядке нету зависимых инструкций. Все остальные инструкции ожидают на стадии R, до разрешения зависимостей

Особенности организации регистровых файлов



- Для выборки целочисленных данных используется рабочий регистровый файл
- В микроархитектуре есть два регистровых файла: рабочий и архитектурный
- Рабочий RF используется для хранения промежуточных результатов работы команд
- Данные в архитектурный RF записываются на последней стадии конвейера D. В случае сброса конвейера данные из архитектурного регистра перебрасываются в рабочий. Этот механизм похож на переименование регистров при спекулятивном исполнении
- Для вещественных инструкций существует только один регистровый файл

Стадии исполнения



- На стадии E для разных типов инструкций происходят разные действия
  - арифметические инструкции вычисляют свои результаты и через bypass передаются на R стадию, для записи их в рабочий RF, и передачи инструкциям, которые их ожидают. Таким образом, латентность арифметических инструкций равна 1
  - условные инструкции вычисляют свои предикаты, и генерируется результат о верном или неверном предсказании
  - инструкции загрузки и сохранения вычисляют свой эффективный адрес и передают его блокам TLB и I-кэш данных
  - кэш данных начинает выборку множества на основе виртуального адреса
  - вещественные инструкции выбирают данные из регистров
- На стадии C приходят данные из I-кэш вместе с тегами
  - закачивается преобразование адресов на TLB

- запускается на выполнение вещественные инструкции

- На стадии M происходит сравнение тегов с физическим адресом, выясняется был ли кэш промах или нет

Стадии исполнения (продолжение)



- На стадии W производятся записи загруженных данных в рабочий RF. Также на этом этапе производятся записи в очередь записи
- На этапе X закачивается обработка большинства вещественных инструкций
- Стадия T отвечает за выделение и генерацию исключений
- Стадия D отвечает за запись результатов в архитектурный регистровый файл

Спекуляция



- Запуск инструкций на исполнение происходит в спекулятивном режиме, так как основан на данных предсказатель ветвления предсказатели ветвления
- Запуск инструкций, которые зависят от инструкций загрузки происходит в спекулятивном режиме в расчете на то что данные будут находиться в кэше первого уровня
- Для поддержки спекуляции процессор имеет два регистровых файла

Сброс конвейера



- Причины:
  - ошибка в предсказании ветвления
  - кэш промах. При кэш промахе все инструкции, которые предшествуют инструкции загрузки, продолжают исполняться, а которые идут после сбрасываются. Инструкция загрузки ожидает получения данных на стадии R.
- Правила при сбросе конвейера
  - инструкция, вызвавшая сброс переносится на стадию R
  - данные из архитектурного RF переносятся в рабочий RF
  - счетчик команд устанавливается на следующую инструкцию за вызвавшей собой инструкцией
  - все инструкции выбираются повторно



СПЕКУЛЯТИВНЫЙ СУПЕРСКАЛЯРНЫЙ ПРОЦЕССОР

Что быстрее динамического суперскалярного процессора?



- динамический суперскалярный процессор (например, алгоритм Томасуло) использует параллелизм инструкций только в одном базовом блоке
- следующим шагом является процессор, который использует параллелизм нескольких базовых блоков

Спекулятивный суперскалярный процессор (ССП)



- Спекуляция - выполнение некоторой операции, необходимость выполнения которой носит вероятностный характер
- ССП способен выполнять инструкции, которые следуют за невыполненной инструкцией условного перехода, основываясь на данных предсказателя ветвления, и в случае неверного предсказания выполнить откат

Этапы исполнения инструкции



Алгоритм Томасуло

- выборка инструкции
- планирование инструкции
- ожидание готовности операндов
- исполнение
- сохранение результата

ССП

- выборка инструкции
- планирование инструкции
- ожидание готовности операндов
- исполнение
- запись результата
- завершение (commit)

Стадии записи результата и завершения



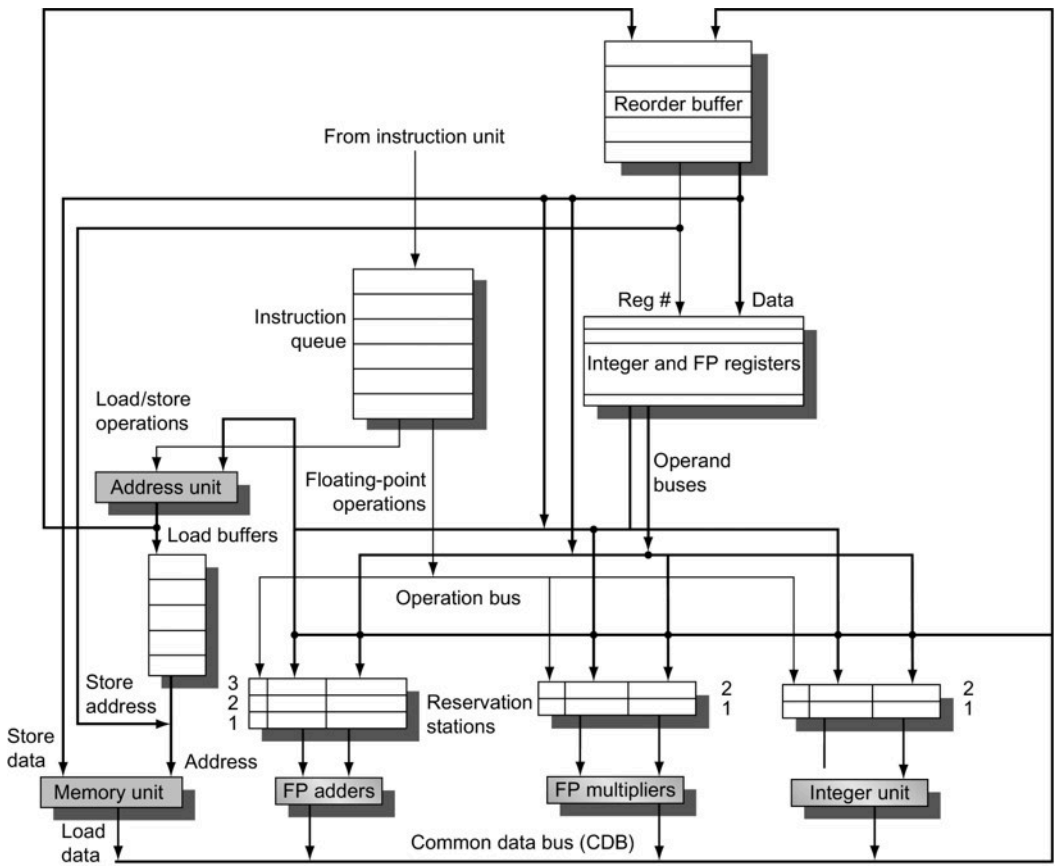
- Стадия записи результата:
  - результат работы инструкции помещается в промежуточное хранилище, а не в основной регистровый файл
- Стадия завершения:
  - проверка корректности исполнения инструкции
  - генерируются исключения, если необходимо
  - записывается результат из временного хранилища в основной регистровый файл

Изменения в аппаратной схеме



- добавлен: буфер упорядочивания (reorder buffer)
  - используется для:
    - хранения результатов работы инструкций между стадией записи результата и стадией завершения
    - поддержки программного порядка инструкций на стадии завершения
  - Элемент # содержит:
    - выполняемую инструкцию: арифметическая, загрузка, сохранение, условный переход
    - адрес результата инструкции
    - поле для временного хранения результата инструкции
- удален: буфер сохранения

Схема ССП


Reorder buffer
From instruction unit
Instruction queue
Reg #
Data
Integer and FP registers
Load/store operations
Address unit
Floating-point operations
Operand buses
Load buffers
Operation bus
Store address
Store data
Address
Reservation stations
FP adders
FP multipliers
Integer unit
Memory unit
Load data
Common data bus (CDB)

Описание работы схемы. Этап планирования



- планирование инструкций осуществляется с вершины очереди инструкций
- для выполнения планирования должны быть доступны RS на требуемом FU и ячейка на #, если хотя бы один из ресурсов не доступен, инструкция ожидает в очереди
- выборка операндов состоит из трех стадий:
  - проверяются есть ли данные в регистровом файле, если есть переписываются в RS, если нет, то
  - проверяются, есть ли данные в #, если есть переписываются в RS, если нет, то
  - в RS записывается номер ячейки #, которая содержит инструкцию, которая вычислит этот операнд
- в файл переименования для результирующего регистра инструкции помещается номер ячейки #, которая ее содержит

Описание работы схемы. Этап исполнения и записи результата



- инструкции запускаются на исполнении по мере готовности их операндов и доступности FU
- по общей шине передается результат инструкции и номер ячейки #, которая ее содержит
- запись результата выполняется одновременно во все ожидающие RS и #
- инструкции загрузки выполняется в две стадии:
  - вычисление адреса
  - чтение из памяти. Запускается, если нет конфликтов с предыдущими инструкциями загрузки и сохранения
- инструкции сохранения на этапе исполнения вычисляет адрес операнда и ожидает готовности результата, запись в память произойдет на стадии завершения

Описание работы схемы. Этап завершения



- инструкции после вычисления результата ожидает в # своего завершения
- завершаются инструкции, которые находятся на вершину #. # поддерживает поведение очереди FIFO
- на этапе завершения:
  - для арифметических инструкций и инструкций загрузки проверяется наличие исключений и если исключений нет, результат записывается в регистровый файл
  - для инструкции сохранения проверяется наличие исключений и если исключений нет, результат записывается в память
  - для инструкции условного перехода проверяется правильно ли она была предсказана или нет, если да, инструкция удаляется из #, если нет, происходит сброс процессора: очищаются все стадии, на планирование отправляется первая инструкция из правильной ветки

Пример 1



1. L.D F6,34(R2)
2. L.D F2,45(R3)
3. MUL.D F0,F2,F4
4. SUB.D F8,F2,F6
5. DIV.D F10,F0,F6
6. ADD.D F6,F8,F2

- Допущения
- Латентность загрузки/сохранения: 1
- Латентность сложения/вычитания: 2
- Латентность умножения: 6
- Латентность деления: 10
- RS для загрузки/сохранения: 2
- RS для простых арифм. опер.: 3
- RS для сложных арифм. опер.: 2

Name	Reservation stations							
	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	no							
Load2	no							
Add1	no							
Add2	no							
Add3	no							
Mult1	no	MUL.D	Mem[45+Regs[R3]]	Regs[F4]			#3	
Mult2	yes	DIV.D		Mem[34+Regs[R2]]	#3		#5	

	Reorder buffer					
Entry	Busy	Instruction		State	Destination	Value
1	no	L.D	F6,34(R2)	Commit	F6	Mem[34+Regs[R2]]
2	no	L.D	F2,45(R3)	Commit	F2	Mem[45+Regs[R3]]
3	yes	MUL.D	F0,F2,F4	Write result	F0	#2 x Regs[F4]
4	yes	SUB.D	F8,F6,F2	Write result	F8	#1 – #2
5	yes	DIV.D	F10,F0,F6	Execute	F10	
6	yes	ADD.D	F6,F8,F2	Write result	F6	#4 + #2

	FP register status									
Field	F0	F1	F2	F3	F4	F5	F6	F7	F8	F10
Reorder #	3						6		4	5
Busy	yes	no	no	no	no	no	yes	...	yes	yes

Что произойдет, если будет сгенерировано исключение?



- если в процессе завершения инструкции выяснится, что она сгенерировала исключение, произойдет сброс конвейера, как при ошибке предсказания перехода

Что обеспечивает выполнение критериев корректного исполнения программы?



- сохранения потока данных происходит за счет использования RS и #, для восстановления явных связей между инструкциями по данным
- сохранения поведения исключений происходит:
  - за счет использования отложенной генерации исключений. Исключение генерируется не на стадии исполнения, когда обнаруживается, а на стадии завершения
  - за счет откатки инструкции в программном порядке

Пример 2. Сравнение Томасуло и ССП



- Линирует 2 инструкции за такт.
- Содержит:
  - устройство выборки сохранения (латентность 2),
  - арифметическое ALU (латентность 1),
  - ALU для вычисления адреса (латентность 1),
  - вещественное ALU (латентность 3),
  - две общие шины для передачи результатов от FU к RS.

1. L.D F0,0(R1)
2. ADD.D F4,F0,F2
3. S.D F4,0(R1)
4. DADDIU R1,R1,#−8 BNE R1,R2,LOOP

Работа Томасуло

Iter. #	Instructions	Issues at clock cycle #	Executes at clock cycle #	Memory access at clock cycle #	Write CDB at clock cycle #	Comment
1	LW R2,0(R1)	1	2	3	4	First issue
1	DADDIU R2,R2,#1	1	5		6	Wait for LW
1	SW 0(R1),R2	2	3	7		Wait for DADDIU
1	DADDIU R1,R1,#4	2	3		4	Execute directly
1	BNE R2,R3,LOOP	3	7			Wait for DADDIU
2	LW R2,0(R1)	4	8	9	10	Wait for BNE
2	DADDIU R2,R2,#1	4	11		12	Wait for LW
2	SW 0(R1),R2	5	9	13		Wait for DADDIU
2	DADDIU R1,R1,#4	5	8		9	Wait for BNE
2	BNE R2,R3,LOOP	6	13			Wait for DADDIU
3	LW R2,0(R1)	7	14	15	16	Wait for BNE
3	DADDIU R2,R2,#1	7	17		18	Wait for LW
3	SW 0(R1),R2	8	19	20		Wait for DADDIU
3	DADDIU R1,R1,#4	8	14		15	Wait for BNE
3	BNZ R2,R3,LOOP	9	19			Wait for DADDIU

Работа ССП

Iter. #	Instructions	Issues at clock #	Executes at clock #	Read access at clock #	Write CDB at clock #	Com-mits at clock #	Comment
1	LW R2,0(R1)	1	2	3	4	5	First issue
1	DADDIU R2,R2,#1	1	5		6	7	Wait for LW
1	SW 0(R1),R2	2	3			7	Wait for DADDIU
1	DADDIU R1,R1,#4	2	3		4	8	Commit in order
1	BNE R2,R3,LOOP	3	7			8	Wait for ADDDI
2	LW R2,0(R1)	4	5	6	7	9	No execute delay
2	DADDIU R2,R2,#1	4	8		9	10	Wait for LW
2	SW 0(R1),R2	5	6			10	Wait for DADDIU
2	DADDIU R1,R1,#4	5	6		7	11	Commit in order
2	BNE R2,R3,LOOP	6	10			11	Wait for DADDIU
3	LW R2,0(R1)	7	8	9	10	12	Earliest possible
3	DADDIU R2,R2,#1	7	11		12	13	Wait for LW
3	SW 0(R1),R2	8	9			13	Wait for DADDIU
3	DADDIU R1,R1,#4	8	9		10	14	Executes earlier
3	BNE R2,R3,LOOP	9	11			14	Wait for DADDIU

Ограничения спекуляции



- если инструкция в спекулятивном режиме вызвала кэш промах, стоит его обрабатывать или нет?
  - обрабатываются обычно только самые легкие события, такие как промах в кэше первого уровня, остальные события откладываются до выхода инструкции из спекулятивного режима
- ошибочная спекуляция приводит к выполнению бессмысленной работы и трате энергии
  - количество инструкций условного перехода над которыми выполняется спекуляция может быть ограничена

Ограничения планирования инструкций условного перехода



- за один такт может быть запланирована только одна инструкция условного перехода, которая предсказана как выполненная
  - объясняется тем, что только на следующем такте будут доступны инструкции из выбранной ветки
- если инструкция предсказана как пропущенная, одновременно с ней могут быть запланированы последующие инструкции и предсказана следующая инструкция условного перехода
- указанное ограничение является серьезным фактором, который ограничивает рост производительности ССП при увеличении количества одновременно завершаемых инструкций

Альтернативный подход к ССП - Переименование регистров



- вычисленные значения инструкций до стадии завершения хранятся в основном регистровом файле во внеархитектурных регистрах, вместо #
- на стадии завершения нужный теневой регистр переименовывается в архитектурный
- такой подход позволяет упростить структуру процессора, так как все данные хранятся в одном месте

Особенности переименования регистров



- требуется два файла переименования для связи архитектурных регистров с физическими
  - первый файл содержит информацию о текущих архитектурных регистрах, их отображении на физические регистры и ячейки #
  - второй файл содержит информацию о действительных архитектурных регистрах, их отображении на физические регистры



АРХИТЕКТУРА VLIW

Как еще можно повысить производительность процессора?



No Title

- За счет роста частоты
- За счет увеличения количества одновременно обрабатываемых команд
  - Instruction Per Cycle

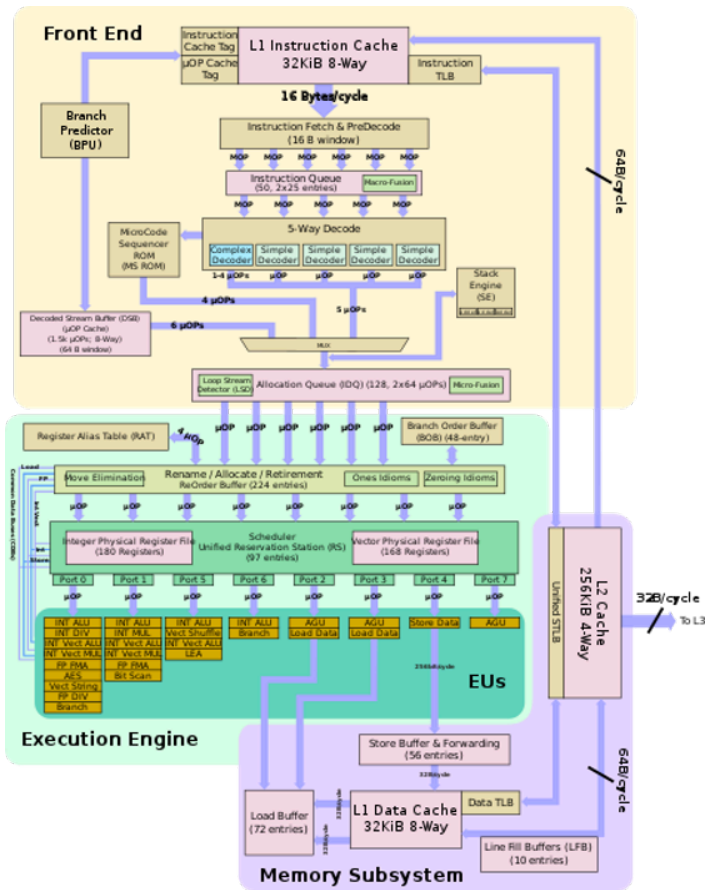


Рис. 43. Intel Skylake

VLIW (Very Long Instruction Word)



- Архитектура, которая специально разработана для повышения количества одновременно исполняемых инструкций
- при одинаковом количестве одновременно исполняемых инструкций VLIW архитектура позволяет создать значительно более простой процессор, чем RISC архитектура

VLIW (Very Long Instruction Word)

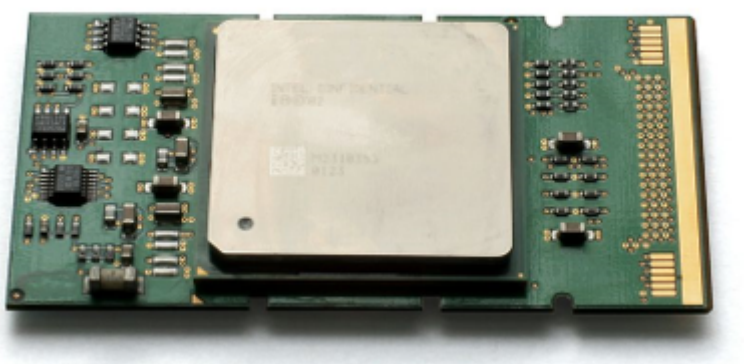
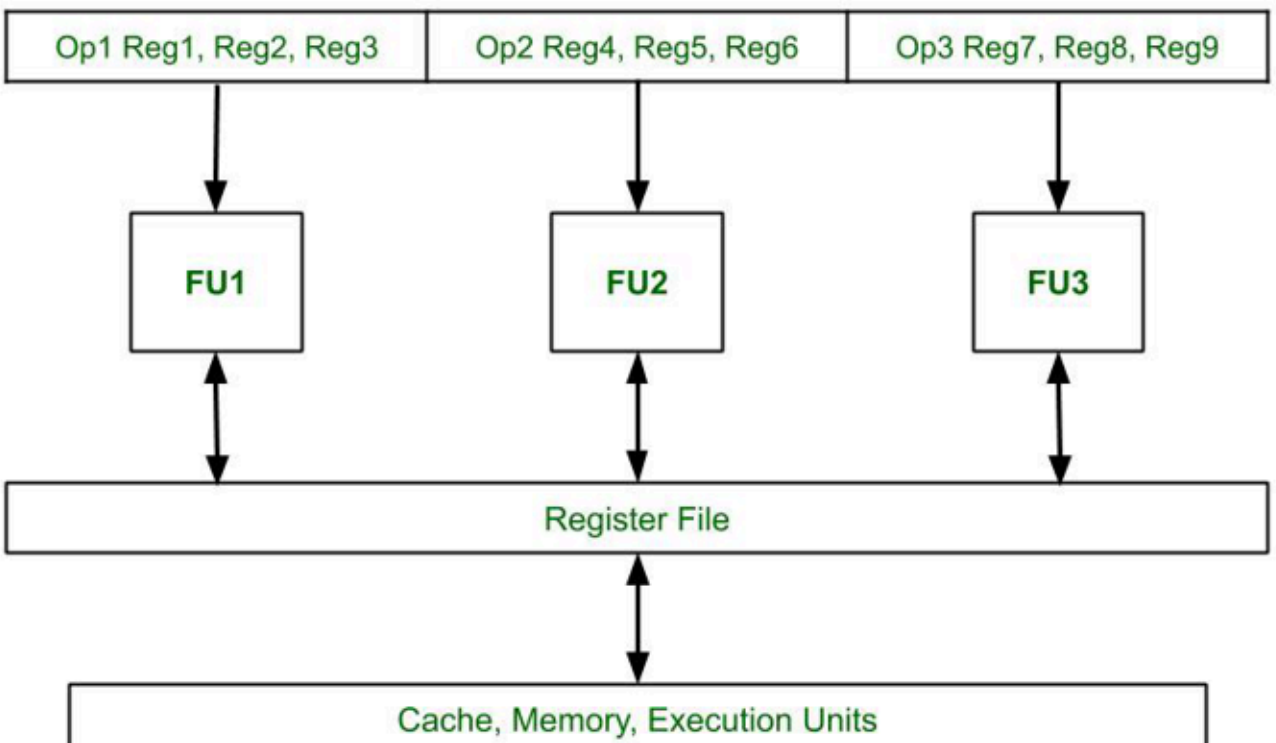


Рис. 44. Intel Itanium 2



Рис. 45. Эльбрус 4С

Ключевая идея



Ключевая идея (продолжение)



- множество SIMD инструкций группируются в пакет (VLIW инструкцию)
- на одном такте выбирается пакет
- каждая инструкция в пакете соответствует строго определенному функциональному устройству (FU)
- между всеми FU делится общий регистровый файл
- типовая длина пакета - 64-128 бит (в зависимости от числа FU)
- проверка зависимостей и планирование исполнения возлагается на компилятор

Плюсы и минусы



- Плюсы:
  - Аппаратная сложность уменьшена
    - отсутствуют блоки проверки зависимостей
    - существенно упрощены блоки планирования исполнения и декодирования инструкций
    - как следствие, сниженное энергопотребление и повышение тактовых частот
- Минусы:
  - высокие требования к компилятору
  - увеличенный размер программного кода
    - более высокие требования к пропускной способности памяти и регистрового файла



- незапланированные события (например, кэш-промах) приводят к останову конвейера
- нерациональное использование пропускной способности памяти и функциональных устройств (например, операция nop)

Исторические предпосылки



История создания



- в начале 80-х Гордон Фишер разработал алгоритм trace планирования, который позволил создать VLIW архитектуру и эффективный компилятор для нее
- в 1984 им была основана компания Multiflow, которая в 1986 выпустила первый компьютер с VLIW архитектурой
  - длина пакета инструкций составляла 1024 бита
  - количество инструкций в одном пакете 28
- в начале 90-х Гордон Фишер перешел в компанию HP, где возглавил отдел исследований перспективных архитектур

Пример плана исполнения (программы) для VLIW процессора



- Процессор состоит из 4 устройств:
  - 2 устройства загрузки/сохранения
  - 1 ALU
  - 1 устройство обработки ветвлений
- Латентность устройств:
  - устройство загрузки/сохранения: 2
  - ALU: 1
- Реализуемая операция: C = A + B

Ld/St unit 0	**Ld/St unit 1**	**Integer ALU**	**Branch Unit**
Load R1,A	Load R2,B	nop	nop
nop	nop	nop	nop
nop	nop	Add R3,R1,R2	nop
Store C,R3	nop	nop	nop

План исполнения



- Содержит информацию
  - об инструкции для каждого FU на каждый такт
  - о моменте запуска инструкции
  - о бездействии устройства (инструкция nop)
- Составляется компилятором
  - компилятор должен знать о качественном и количественном составе FU, об их латентности

Статический VLIW процессор



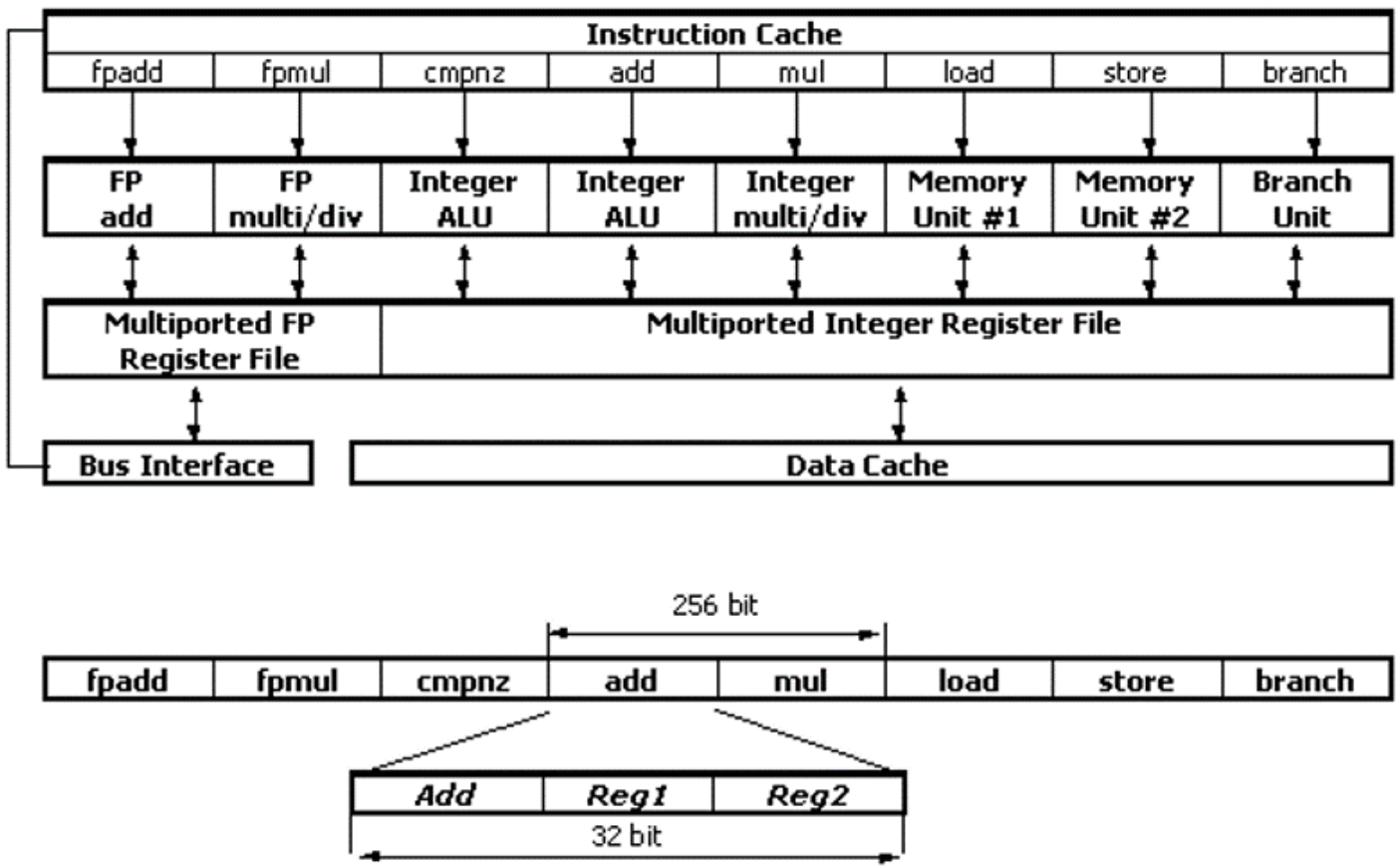
- Особенность: за запуск инструкций отвечает компилятор
  - компилятор должен правильно расставить nop-ы
    - если инструкция пойдет на исполнение до того момента, как предыдущая инструкция закончит произойдет сбой конвейера
    - как быть с устройствами со случайной латентностью (например, устройство загрузки/сохранения при наличии кэша)
    - компилятор всегда выбирает максимальную латентность для устройств со случайной латентностью
- Недостатки:
  - неэффективность кэша
  - очень большой размер программы из-за большого количества nop инструкций

Динамический VLIW процессор



- Особенность: за запуск инструкций отвечает процессор
  - процессор ожидает готовности данных для всех инструкций в пакете
  - только после этого запускает пакет
- Позволяет эффективно использовать кэш
- Уменьшает размер программы

Пример внутренней организации



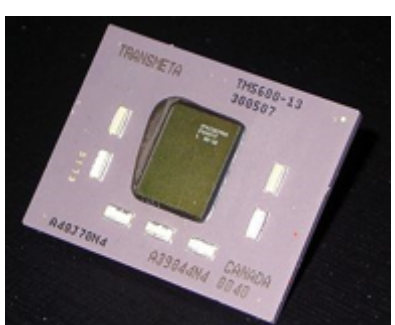
Большие проблемы VLIW архитектур



- большой размер скомпилированной программы
- отсутствие бинарной совместимости
  - изменение количественного и качественного состава FU процессора приводит к бинарной несовместимости с предыдущей версией
  - код нужно перекомпилировать для каждого процессора

Область применения VLIW процессоров

- основная область применения - DSP процессоры
  - малый объем программного обеспечения позволяет его относительно просто перекомпилировать
- использование VLIW позволяет упростить процессор, уменьшить его энергопотребление, увеличить производительность
- Пример процессоров:
  - STMicroelectronics ST200, TI C6xx, Analog Devices SHARC
  - Philips TriMedia
  - Transmeta Crusoe





EPIC И АРХИТЕКТУРА IA-64

Архитектура EPIC



- Прямой наследник VLIW

EPIC (explicit parallel instruction set)



- Призвана сохранить достоинства VLIW архитектуры и устранить ее недостатки
- Общие черты VLIW и EPIC:
  - возможность параллельного исполнения инструкций задается на уровне архитектуры, а не определяется динамически в процессе исполнения
- Преимущества EPIC:
  - решена проблема бинарной совместимости между различными процессорами

История



- разработки архитектуры следующего за VLIW поколения начались в компании HP в конце 80-х годов
- в 1993 году разработана первая EPIC архитектура PlayDoh
- в 1997 году анонсирована архитектура Intel IA-64 (компании Intel и HP)
- в 2001 выпущен первый процессор Intel Itanium
- в 2002 выпущен процессор Intel Itanium 2

Сравнение VLIW и EPIC



- VLIW:
  - параллельное выполнение инструкций задается в рамках одного пакета
  - количество инструкций в одном пакете определяется количеством исполняемых устройств в процессоре
- EPIC:
  - параллельно исполняемые инструкции задаются специальными шаблонами в пакете
  - количество параллельно исполняемых инструкций может быть больше количества инструкций в пакете
  - количество инструкций в пакете определяется архитектурой и не зависит от аппаратной реализации

Проблема бинарной совместимости



- проблема устраняется за счет того, что задача назначения инструкций на FU **выполняется процессором**
  - это позволяет устранить зависимость количества инструкций в пакете от количества исполняемых устройств в процессоре

Пакет инструкций



- состоит из слотов инструкций и шаблона
- шаблон определяет
  - тип инструкций в слотах
  - расположение разделителя последовательности инструкций
- инструкции в последовательности являются независимыми
  - запуск инструкций из последовательности начинается, когда конфликты для всех инструкций разрешены
  - запускаются процессором в произвольном порядке по мере готовности устройств
  - после запуска всех инструкций из последовательности процессор переходит к обработке следующей последовательности

Типы слотов и типы инструкций (IA-64)



Execution unit slot	Instruction type	Instruction description	Example instructions
I-unit	A	Integer ALU	Add, subtract, and, or, compare
	I	Non-ALU	integer Integer and multimedia shifts, bit tests, moves
M-unit	A	Integer ALU	Add, subtract, and, or, compare
	M	Memory access	Loads and stores for integer/FP registers
F-unit	F	Floating point	Floating-point instructions
B-unit	B	Branches	Conditional branches, calls, loop branches
L + X	L + X	Extended	Extended immediates, stops and no-ops

Шаблоны пакетов IA-64

Template	Slot 0	Slot 1	Slot 2
0	M	I	I
1	M	I	I
2	M	I	I
3	M	I	I
4	M	L	X
5	M	L	X
8	M	M	I
9	M	M	I
10	M	M	I
11	M	M	I
12	M	F	I
13	M	F	I
14	M	M	F
15	M	M	F
16	M	I	B
17	M	I	B
18	M	B	B
19	M	B	B
22	B	B	B
23	B	B	B
24	M	M	B
25	M	M	B
28	M	F	B
29	M	F	B

Пример программы



LOOP:

- L.D R2,0(R1)
- ADD.D R5,R2,R3
- SUB.D R6,R2,R4
- S.D R5,0(R1)
- S.D R6,0(R7)
- DADDIU R1,R1,#−8
- DADDIU R7,R7,#−8
- BNE R1,R8,LOOP

0	M	I	I
0	L.D R2,0(R1)	nop	nop
0	M	I	I
0	nop	ADD.D R5,R2,R3	SUB.D R6,R2,R4
9	M	M	I
9	S.D R5,0(R1)	S.D R6,0(R7)	nop
2	M	I	I
2	nop	DADDIU R1,R1,#−8	DADDIU R7,R7,#−8
23	B	B	B
23	BNE R1,R8, 0	nop	nop

Спекуляция по управлению



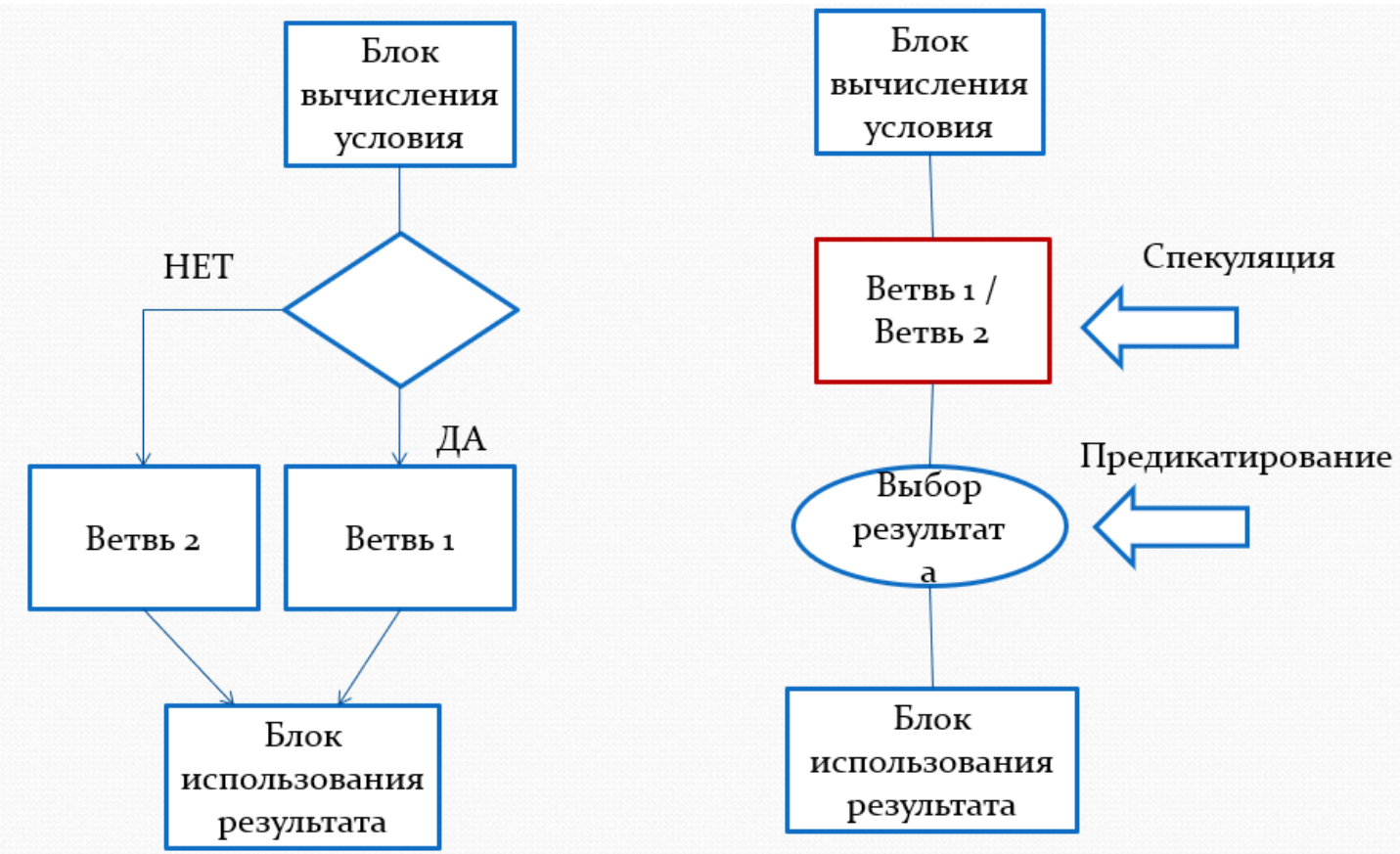
- Спекуляция по управлению - это возможность корректно исполнить инструкции в ветке, для которой еще не вычислено условие
- процессоры с архитектурой RISC и CISC выполняют спекуляцию на аппаратном уровне. Первоначально такой возможности в этой архитектуре не закладывалось
- поддержка спекуляции по управлению включена в состав архитектуры EPIC

Спекуляция по управлению в EPIC



- Спекуляция по управлению в EPIC основана на:
  - наличие двух типов инструкций: спекулятивных и не спекулятивных
  - параллельном исполнении спекулятивных инструкций из двух веток
  - использовании предикатирования для выбора результата корректной ветки
  - использовании третьего состояния регистров для сохранения поведения исключений во время спекуляции

Предикатирование и спекуляция



Генерация исключений



- все исключения генерируемые спекулятивными инструкциями являются отложенными
- спекулятивная инструкция, сгенерировавшая отложенное исключение, помещает в свой регистр результата значение NaT (Not a Thing)
- не спекулятивные инструкции генерируют обычные исключения

NaT состояние регистра



- NaT состояние является необходимым для поддержки отложенных исключений, так как позволяет их сохранить
- реализуется добавлением дополнительного бита к каждому регистру процессора
- спекулятивными являются только те инструкции, которые сохраняют данные в регистры с поддержкой состояния NaT (универсальные регистры, вещественные регистры)
- не спекулятивные инструкции сохраняют результаты в память или обычные регистры
- если один из аргументов спекулятивной инструкции NaT, результат инструкции тоже NaT
- если один из аргументов не спекулятивной инструкции NaT, генерируется не отложенное исключение

Достоинства и недостатки программной спекуляции по управлению в EPIC



- Достоинства:
  - маленькие аппаратные затраты на реализацию
  - нет затрат на неверное предсказание
- Недостатки:
  - требует исполнения большего количества инструкций
  - проигрывает по скорости аппаратной спекуляции в случае низкой вероятности неверного предсказания перехода

Спекуляция по данным (задача)



- спекулятивный суперскалярный процессор может маскировать большую латентность инструкций загрузки за счет внеочередного исполнения других инструкций
- EPIC процессор обрабатывает последовательности независимых инструкций в программном порядке
  - поэтому не может пропустить зависимые инструкции
- чтобы скрыть латентность инструкций загрузки, он должен выполнять их как можно раньше
  - компилятор должен перенести инструкцию загрузки как можно выше по коду относительно инструкции, которая от нее зависит

Спекуляция по данным (продолжение)



- при переносе инструкции загрузки вверх по коду компилятор должен сохранять поток данных
  - нельзя менять местами инструкции загрузки и сохранения, которые выполняются по одному адресу (!!!!!)
- встречая код:
  - st a(r1), r2
  - ld r3, a(r5)
  - компилятор должен проверить совпадают ли a(r1) и a(r5) для того, чтобы переставить две инструкции местами
- в общем случае задача на стороне компилятора не решается (!!!)
  - решается на стороне процессора

Спекуляция по данным



- компилятор переносит инструкцию настолько вверх, насколько нужно, не обращая внимание на возможные конфликты
- перед инструкцией потребителем данных вставляется специальная инструкция проверки, которая проверяет, были ли сохранения по целевому адресу после запуска инструкции загрузки
- если произошел конфликт, инструкция проверки может сгенерировать исключение или выполнить обычную загрузку

Архитектура IA-64



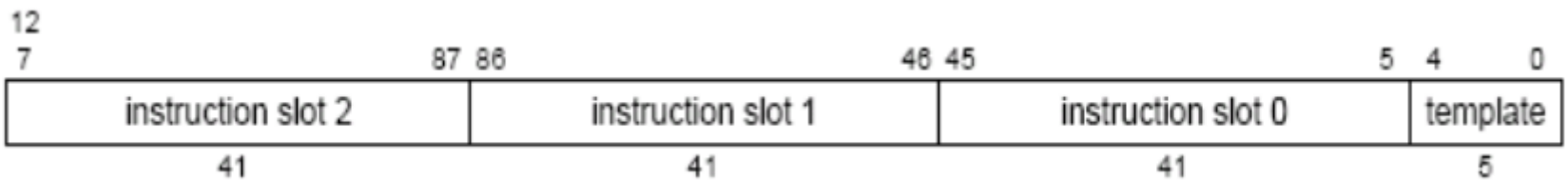
- единственный коммерчески успешный представитель EPIC архитектур
  - поддерживается только процессором Intel Itanium

Регистровый файл



- 128 универсальных 64-х битных регистров общего назначения с поддержкой третьего состояния
  - первые 32 регистра формируют классический регистровый файл
  - остальные 96 формируют динамический оконный регистровый файл
- 128 вещественных 80-и битных регистров с поддержкой третьего состояния
- 64 1-битных регистра предиката
- 8 64-х битных регистров для хранения адресов перехода

Кодирование пакета инструкций



- Пакет инструкций имеет размер 128 бит:
  - 3 слота по 41 биту каждый
  - 5 бит отведено на шаблон пакета
- инструкция может занимать либо 1, либо 2 слота

Кодирование инструкции



- классическая трехадресная инструкция имеет следующий формат:

40	37	36	27	26	20	19	13	12	6	5	0
opCode	opCode	Ext opCode	Ext opCode	R3	R3	R2	R2	R1	R1	qp	qp

- Формат:
  - поле кода операции
  - три операнда
  - поле предиката
    - указывает на один из регистров предикатов, который определяет, будет ли исполняться инструкция или нет

Инструкции загрузки



- существует 4 типа инструкций загрузки:
  - простые
  - спекулятивные. Генерируют отложенные исключения
  - расширенные. Поддерживают спекуляцию по данным
  - расширенные спекулятивные. Поддерживают как спекуляцию по данным, так и по управлению

Инструкции сравнения и условного перехода



- инструкции сравнения сравнивает два регистра по условию и результат записывается в два регистра предикатов
  - прямое значение
  - инверсное значение
- инструкции перехода выполняют переход на первую инструкцию пакета
  - условие задается регистром предикатов